

Secure Android Automotive OS on Raspberry Pi 5

A Project Report

submitted by

GANESH H

CS23M510

*in partial fulfilment of the requirements
for the award of the degree of*

MASTER OF TECHNOLOGY



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY, MADRAS.**

June 2026

THESIS CERTIFICATE

This is to certify that the thesis entitled **Secure Android Automotive OS on Raspberry Pi 5**, submitted by **Ganesh H**, to the Indian Institute of Technology, Madras, for the award of the degree of **Master of Technology**, is a bona fide record of the research work carried out by him under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Prabhakara Pediredla

Research Guide

Principal/Manager Engineer

Qualcomm India Pvt. Ltd.

Bengaluru, 560 066

Place: Bengaluru

Date: June 9, 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my guide, Prabhakara Pediredla, Principal/Manager Engineer at Qualcomm India Pvt. Ltd., for his invaluable guidance right from the initial setup of the development environment through each step of the implementation. His practical insights into embedded systems security and his continued availability throughout the course of this project were instrumental in bringing this work to completion.

I would also like to thank my wife for her constant encouragement, patience, and unwavering support throughout this journey.

ABSTRACT

Keywords: Android Automotive OS, Raspberry Pi 5, Android Verified Boot, SELinux, File-Based Encryption, TARA, ISO/SAE 21434, U-Boot, Secure Boot

The automotive industry’s shift towards Software-Defined Vehicles has elevated Android Automotive OS (AAOS) as a leading in-vehicle infotainment platform. However, deploying a security-hardened AAOS on commodity single-board computers — specifically the Raspberry Pi 5 (BCM2712, quad-core ARM Cortex-A76, AArch64) — introduces significant engineering challenges: the platform lacks hardware-backed security enclaves, native AVB support, and Android-aware firmware.

This project evaluates the feasibility and correctness of running a security-hardened AAOS on RPi5 hardware by enabling and assessing three Android security subsystems: Android Verified Boot 2.0 (AVB2), SELinux Enforcing mode, and File-Based Encryption 2.0 (FBE). A reproducible build pipeline, `meta-rpi5-secure-aosp`, was developed to automate the full workflow — `sync` → `patch` → `build` → `sign` → `sdcard` — with per-stage and per-component granularity.

AVB2 was enabled through custom U-Boot patches and an external signing stage using `avbtool`, covering `system` and `vendor` partitions with SHA-256/RSA-4096 and a fail-closed policy. SELinux Enforcing mode was validated with a correctly scoped policy. FBE 2.0 was enabled using AES-256-XTS on `userdata` and metadata encryption, supported by a dedicated `metadata` partition. The full pipeline was validated on real RPi5 hardware via UART at 115200 baud.

A formal Threat Analysis and Risk Assessment (TARA) aligned with ISO/SAE 21434 was conducted, covering the full boot chain and all attack surfaces. The risk register comprises 23 threat scenarios — 9 Critical, 6 High, 8 Medium, 1 Low — with a mitigation mapping for all High/Critical risks and an explicit residual risk register of 10 accepted risks, bounded by the hardware constraints of a commodity development platform.

Contents

ACKNOWLEDGEMENTS	ii
ABSTRACT	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
ABBREVIATIONS	x
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Contributions	3
1.5 Report Structure	3
2 BACKGROUND	5
2.1 Android Automotive OS	5
2.2 Android Security Architecture	6
2.3 Raspberry Pi 5 Hardware	7
2.4 U-Boot Bootloader	8
2.4.1 U-Boot AVB Integration	8
2.5 Android Verified Boot 2.0 (AVB)	9
2.5.1 VBMeta Descriptors	9
2.5.2 Rollback Protection	10
2.5.3 Device States and Boot Colour Model	10
2.5.4 A/B Slot Support	11
2.5.5 libavb and avbtool	12
2.6 dm-verity	12
2.7 Full-Disk Encryption (FDE)	13
2.7.1 Key Derivation Chain	13

2.7.2	Boot Flow and Service Classes	13
2.7.3	Limitations and Deprecation Rationale	14
2.8	File-Based Encryption (FBE)	14
2.9	SELinux on Android	15
2.10	Android Init System	15
2.11	ISO/SAE 21434 and Threat Analysis and Risk Assessment (TARA)	15
2.11.1	Asset Identification	15
2.11.2	STRIDE Threat Modelling	15
2.11.3	Risk Scoring	16
2.11.4	Risk Treatment and Residual Risk	16
3	SYSTEM ARCHITECTURE	17
3.1	Overview	17
3.2	Boot Chain	17
3.3	Partition Layout	19
3.4	Build Pipeline	20
3.4.1	Stages	20
3.4.2	Configuration Model	20
3.5	Security Design Decisions	21
3.5.1	External Signing Pipeline	21
3.5.2	Fail-Closed AVB Policy	21
3.5.3	FBE Dependency Chain	21
4	IMPLEMENTATION	22
4.1	U-Boot Patch: PL011-AXI Serial Driver	22
4.1.1	Problem	22
4.1.2	Solution	22
4.1.3	Risk Assessment	23
4.2	U-Boot AVB Kconfig Configuration	23
4.2.1	Public Key Injection	24
4.3	U-Boot Boot Script	24
4.4	Kernel Patch: Boot-Layer Diagnostic Markers	24
4.5	AOSP Patch: AVB-Aware fstab Selection	25
4.6	AOSP Patch: SELinux Enforcing Mode	25
4.6.1	Policy Structure	25
4.6.2	Android 16 Enforcing Behaviour	26
4.6.3	Configuration	26
4.6.4	AVC Denial Handling	26
4.6.5	Serial Log Evidence	27
4.7	AOSP Patch: File-Based Encryption	28

4.8	FBE Bring-up: Metadata Partition SELinux Boot Loop	29
4.8.1	Symptom	29
4.8.2	Diagnosis: Two Concurrent Root Causes	29
4.8.3	Fix: Replace <code>mkfs.ext4</code> with <code>e2fsdroid</code>	30
4.8.4	Security Significance	30
4.9	AOSP Patch: Boot Diagnostics Service	31
4.10	AVB Signing Stage	31
4.11	Build Orchestrator CLI	32
5	RESULTS AND EVALUATION	33
5.1	Overview	33
5.2	Build Pipeline Execution	33
5.2.1	U-Boot Build	33
5.2.2	AOSP Build	34
5.2.3	Signing Stage	34
5.2.4	SD Card Image Assembly	34
5.3	Boot Sequence Validation	35
5.3.1	Unsigned Build Validation	35
5.3.2	Signed Build Validation	37
5.4	Evaluation Summary	40
5.5	Limitations	40
6	THREAT ANALYSIS AND RISK ASSESSMENT	43
6.1	Overview	43
6.2	Asset Inventory	43
6.3	Threat Scenarios	44
6.4	Risk Register	46
6.5	Mitigation Mapping	47
6.6	Residual Risk Register	49
6.7	Discussion: Software Security Ceiling on RPi5	51
6.8	Technical Reasoning	52
6.9	External References	52
7	CONCLUSION	54
7.1	Summary	54
7.2	Security Posture Assessment	55
7.3	Technical Reasoning	56
7.3.1	Causal Chain	56
7.3.2	Alternatives Considered	56
7.3.3	Risk and Tradeoffs	56

7.4	Future Work	57
7.5	External References	58

List of Tables

2.1	Project-scope security blocks within the AAOS architecture	7
2.2	SFOP Impact Scale (ISO/SAE 21434 Clause 15)	16
3.1	GPT Partition Layout (signed build, 14.6 GB SD card)	20
4.1	RPi5 ARM64 U-Boot load addresses vs. AVB buffer placement	23
4.2	Kernel boot-layer diagnostic markers	25
5.1	Unsigned build validation results	37
5.2	Signed build validation results	41
5.3	Evaluation against project objectives	41
6.1	Asset inventory summary by category (24 assets total)	44
6.3	Threat scenarios (23 total)	44
6.4	Risk register: Boot chain threats (T-001 to T-008)	46
6.6	Risk register: Build pipeline threats (T-009 to T-013)	47
6.8	Risk register: Runtime interface threats (T-014 to T-017)	47
6.16	Mitigation mapping for Critical and High risks	47
6.10	Risk register: Data confidentiality threats (T-018 to T-020)	49
6.12	Risk register: Configuration and policy threats (T-021 to T-023)	49
6.14	Risk level summary across all 23 threat scenarios	50
6.17	Residual risk register (10 accepted risks)	51

List of Figures

2.1	AAOS system architecture. Six layers from Linux Kernel (bottom) to Applications (top), with automotive-specific components (Vehicle HAL, Car Service, EVS) and security modules highlighted at each layer. Source: [1–3].	6
2.2	AVB with <code>boot</code> , <code>system</code> , and <code>vendor</code> partitions. The <code>vbmeta</code> partition holds signed descriptors for all three. Source: [4].	9
2.3	AVB with a chained partition: the top-level <code>vbmeta</code> embeds the public key used to verify an independently signed partition’s own VBMeta image. Source: [4].	10
2.4	AVB rollback index values embedded in <code>vbmeta</code> and partition images. Source: [4].	10
2.5	Device-stored rollback index locations. Values must be $\geq$ those in the signed metadata for boot to proceed. Source: [4].	11
2.6	AVB with A/B partitions: both <code>_a</code> and <code>_b</code> slot images are independently signed; the bootloader selects and verifies the active slot. Source: [4].	11
2.7	Recommended AVB boot flow: the bootloader verifies the VBMeta signature, checks rollback indices, validates partition hashes/hashtrees, and passes the verified boot state to the kernel via the commandline. Source: [4].	12
3.1	Secure boot chain on Raspberry Pi 5. Three verification gates (AVB, dm-verity, FBE) each enforce a fail-closed policy: any verification failure triggers a device reboot with no fallback to a degraded boot state.	19

ABBREVIATIONS

AAOS	Android Automotive OS
ADB	Android Debug Bridge
AES	Advanced Encryption Standard
AOSP	Android Open Source Project
ARM	Advanced RISC Machine
AVB	Android Verified Boot
AVC	Access Vector Cache
AXI	Advanced eXtensible Interface
CBC	Cipher Block Chaining
CE	Content Encryption
CLI	Command-Line Interface
CTS	Ciphertext Stealing
DE	Device Encryption
DEK	Data Encryption Key
DM	Device Mapper
DT	Device Tree
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FBE	File-Based Encryption
FDE	Full Disk Encryption

FEC	Forward Error Correction
FIT	Flattened Image Tree
GPIO	General-Purpose Input/Output
GPT	GUID Partition Table
HDMI	High-Definition Multimedia Interface
HSM	Hardware Security Module
HVAC	Heating, Ventilation, and Air Conditioning
ISO	International Organization for Standardization
IVI	In-Vehicle Infotainment
KEK	Key Encryption Key
MAC	Message Authentication Code
MMC	MultiMediaCard
OEM	Original Equipment Manufacturer
OTA	Over-The-Air
OTP	One-Time Programmable
PII	Personally Identifiable Information
RPi	Raspberry Pi
RPMB	Replay Protected Memory Block
RSA	Rivest–Shamir–Adleman
SAE	Society of Automotive Engineers
SBC	Single-Board Computer
SDV	Software-Defined Vehicle
SELinux	Security-Enhanced Linux
SFOP	Safety, Financial, Operational, and Privacy
SHA	Secure Hash Algorithm

SoC	System on Chip
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TARA	Threat Analysis and Risk Assessment
TEE	Trusted Execution Environment
TFTP	Trivial File Transfer Protocol
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
XTS	XEX-based Tweaked-codebook mode with ciphertext Stealing
YAML	YAML Ain't Markup Language

Chapter 1

INTRODUCTION

1.1 Motivation

The automotive industry is undergoing a significant transformation driven by the rise of Software-Defined Vehicles (SDVs). Modern vehicles increasingly rely on complex in-vehicle infotainment (IVI) and instrument cluster systems that require secure, updatable, and open software platforms. Android Automotive OS (AAOS) has emerged as a leading choice for automotive-grade IVI systems, offering a well-established ecosystem, rich driver and middleware support, and a structured security model built around Android’s verified boot and encryption framework [3].

The deployment of AAOS on custom embedded hardware introduces significant engineering challenges. The standard AOSP build system and security infrastructure are optimised for certified SoCs with dedicated secure enclaves, re-provisioned fusing, and hardware-backed key storage. Bringing these capabilities to a commodity single-board computer — specifically the Raspberry Pi 5 (RPi5) — requires careful adaptation of the boot chain, partition layout, and security policy.

It also presents us with the engineering challenges of a security-critical embedded system demands i.e. structured security analysis. ISO/SAE 21434 [5] is the automotive cybersecurity engineering standard that mandates Threat Analysis and Risk Assessment (TARA) as a design artifact: identifying assets, applying the STRIDE(Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) threat model, scoring risks using Safety/Financial/Operational/Privacy (SFOP) impact categories, and explicitly documenting accepted residual risks. The conducting of TARA alongside implementation revealed where software-level security protections are sufficient and where they are inherently bounded by hardware constraints.

1.2 Problem Statement

The Raspberry Pi 5, based on the Broadcom BCM2712 SoC (quad-core ARM Cortex-A76, AArch64), is a powerful and affordable development platform, but it wasn't designed with Android Automotive in mind. Several technical gaps must be bridged:

- **Bootloader:** The RPi5 firmware boots directly into the Linux kernel by default. Inserting U-Boot into this chain and enabling Android Verified Boot (AVB2) requires custom U-Boot patches and a signed boot script.
- **AVB integration:** The AVB2 signing pipeline must be performed externally (not via `BOARD_AVB_ENABLE`) because the AOSP build system doesn't natively support the RPi5 target. This requires a custom sign stage that instruments `avbtool` directly.
- **Encryption:** File-Based Encryption (FBE) relies on kernel `fscrypt` support, a correctly formed `fstab`, and a dedicated metadata partition. These are not part of the upstream RPi5 AOSP device tree.
- **SELinux:** Upstream RPi5 AOSP device trees ship with SELinux in Permissive mode, which silently allows all policy violations and provides no runtime enforcement. Enabling Enforcing mode requires a correctly scoped policy and validation that no critical services are denied.
- **Observability:** Diagnosing the multi-layer boot chain across RPi firmware, U-Boot, the Android kernel, and Android init requires instrumentation at each layer. This is absent in the stock sources.

1.3 Objectives

The goal of this project was to evaluate the feasibility and correctness of running a security-hardened Android Automotive OS on commodity Raspberry Pi 5 hardware, by enabling and assessing three Android security subsystems: Android Verified Boot 2.0 (AVB2), SELinux Enforcing mode, and File-Based Encryption 2.0 (FBE). To support reproducible evaluation, a build pipeline — `meta-rpi5-secure-aosp` — was developed. The specific objectives were:

1. **AVB 2.0 evaluation:** Enable a verified boot chain from U-Boot through to Android userspace using AVB2 (SHA-256/RSA-4096 partition signing covering `system` and `vendor`, fail-closed policy), and assess its correctness on RPi5 hardware via per-boot UART diagnostics.

2. **SELinux evaluation:** Enable SELinux Enforcing mode within the AAOS environment on RPi5 and assess policy enforcement behaviour and its impact on system operation.
3. **FBE 2.0 evaluation:** Enable File-Based Encryption (AES-256-XTS on `userdata`, metadata encryption on the `metadata` partition) and validate it functions correctly on the platform.
4. **Reproducible pipeline:** Deliver a structured, automated build pipeline (`sync` → `patch` → `build` → `sign` → `sdcard`) with per-stage and per-component granularity, so that the security configurations above can be reliably reproduced and re-evaluated.
5. **Threat analysis (TARA):** Conduct an ISO/SAE 21434-aligned Threat Analysis and Risk Assessment covering the full boot chain, all attack surfaces, and explicitly documenting accepted residual risks [5].

1.4 Contributions

The main contributions of this work are:

- A U-Boot patch (`serial: pl01x: add arm,pl011-axi compatible`) enabling console output on the RPi5's default UART path.
- A set of AOSP device-tree patches adding AVB-aware `fstab` selection, FBE configuration, SELinux Enforcing mode, and boot diagnostics.
- A kernel patch adding four boot-layer diagnostic markers to `init/main.c`.
- An ISO/SAE 21434-aligned TARA producing a risk register of 23 threat scenarios (9 Critical, 6 High, 8 Medium, 1 Low), a mitigation mapping for all High/Critical risks, and an explicit residual risk register of 10 accepted risks [5].
- Validation of the full pipeline on real RPi5 hardware at 115200 baud via UART.
- A Python orchestration CLI (`rpi5-build`) with a `BuildContext` shared state model, modular stage implementations, and a comprehensive set of configurable security options — developed as a reproducibility artefact to support the above evaluation.

1.5 Report Structure

The rest of this report is structured as follows. Chapter 2 provides background on the key technologies. Chapter 3 describes the overall system architecture and design decisions. Chapter 4 details the implementation of each component. Chapter 5 presents the

evaluation results. Chapter 6 presents the Threat Analysis and Risk Assessment (TARA) including the risk register, mitigation mapping, and accepted residual risks. Chapter 7 concludes with a summary and directions for future work.

Chapter 2

BACKGROUND

This section deals with survey of the key technologies and concepts underpinning the project.

2.1 Android Automotive OS

Android Automotive OS (AAOS) is a full-stack in-vehicle operating system derived from AOSP. Unlike Android Auto — which mirrors a phone onto a head unit — AAOS runs directly on the vehicle’s IVI hardware [3]. It exposes automotive-specific APIs (vehicle properties, HVAC, driving state restrictions) while retaining the full Android framework, making it suitable for both OEM-integrated experiences and third-party application deployment.

AAOS uses the same build system and security primitives as standard AOSP: the `lunch` target, Soong/Make hybrid build rules, and the Android security model (SELinux, AVB, encryption). This project targets the `aosp_rpi5_car-bp4a` lunch target, producing a Bluetooth-primary Automotive variant.

Figure 2.1 shows the full AAOS system architecture across six layers. Security-relevant components are present at every layer — from kernel-level SELinux and dm-verity, through HAL-level KeyMint and DRM, to framework and application-level permission and certificate management.

Not all security blocks in Figure 2.1 are within this project’s scope. Table 2.1 identifies the subset that this project directly configures or enables, distinguishing them from components that are either provided by base AOSP unchanged or absent on the RPi 5 hardware (for example, no Biometric HAL or DRM Widevine is required for a development automotive target).

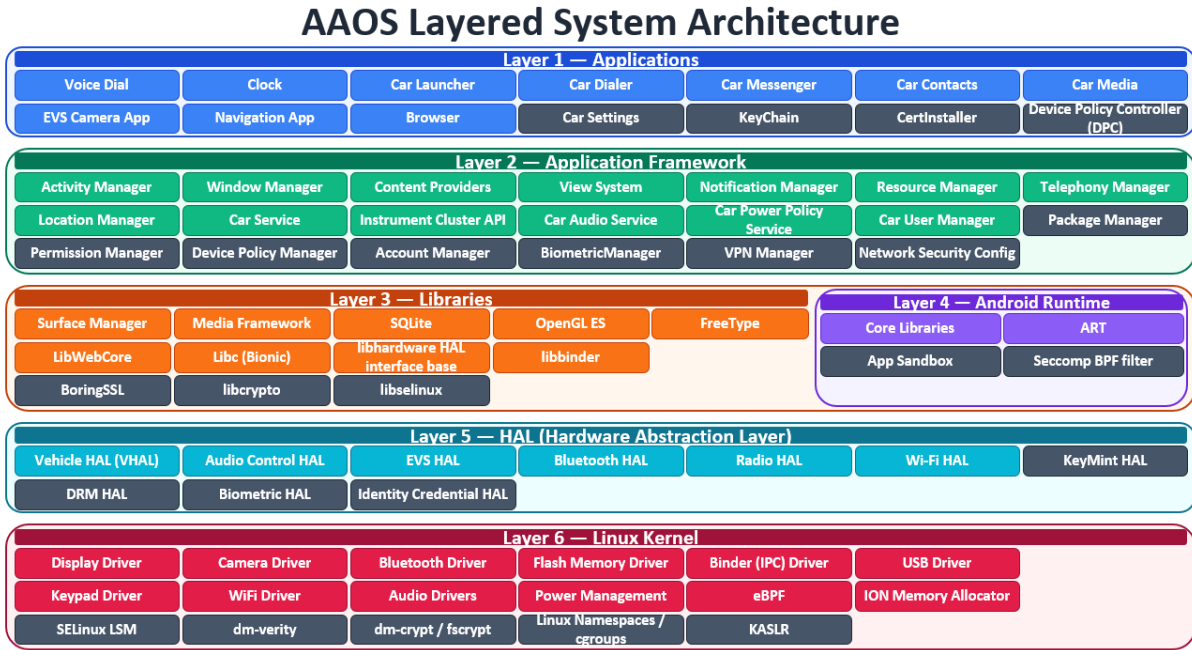


Figure 2.1: AAOS system architecture. Six layers from Linux Kernel (bottom) to Applications (top), with automotive-specific components (Vehicle HAL, Car Service, EVS) and security modules highlighted at each layer. Source: [1–3].

2.2 Android Security Architecture

Android implements a multi-layered security model in which hardware-backed trust mechanisms, OS-level isolation, and application-level controls reinforce one another [2]. The key layers are:

Application Sandbox Each Android application is assigned a unique Unix UID and runs in its own isolated process. The Linux kernel enforces UID-based access controls, preventing one application from directly accessing another’s data or code without explicit inter-process communication.

Verified Boot (AVB) Establishes a hardware-rooted chain of trust from the bootloader through to all verified partitions using cryptographic signatures. Any modification to a verified partition is detected before the OS is allowed to start. (Detailed in Section 2.5.)

SELinux Enforces Mandatory Access Control (MAC) over all processes and files, including those running with root or superuser privileges, constraining the damage any single compromise can cause. (Detailed in Section 2.9.)

Encryption Three complementary mechanisms protect data at rest: File-Based Encryption (FBE, Section 2.8) encrypts each file independently with a per-file key; Full-Disk

Table 2.1: Project-scope security blocks within the AAOS architecture

Layer	Block	Project action
Linux Kernel	SELinux LSM	Enforcing mode enabled; policy validated at runtime
Linux Kernel	dm-verity	AVB2 hashtree applied to system and vendor partitions
Linux Kernel	fscrypt	File-Based Encryption activated on /data
HAL	KeyMint HAL	Used for AVB root-key storage and FBE key wrapping
HAL	Vehicle HAL	Required AAOS component; present via RPi5 device tree

Encryption (FDE, Section 2.7) is the legacy block-level approach; and metadata encryption protects filesystem metadata (directory tree, file sizes, permissions) that FBE does not cover.

Android Keystore Provides cryptographic operations (encryption, signing, key agreement) where the key material is stored inside a Trusted Execution Environment (TEE) or Secure Element, preventing key extraction even from a rooted device [2].

Trusty TEE A minimal secure OS that executes on the same processor as Android but is isolated via hardware mechanisms (ARM TrustZone). It hosts security-sensitive services such as key storage, attestation, and biometric authentication.

Gatekeeper and Weaver Gatekeeper performs PIN / pattern / password authentication inside the TEE and enforces hardware-backed lockout after repeated failures. Weaver provides an additional layer of credential binding in a dedicated secure element where available.

App Signing All Android applications must be cryptographically signed before installation. Signing ties updates to a developer identity and underpins the package manager’s integrity checks.

This project engages four of these layers directly: AVB (boot-time integrity), FBE (user-data encryption), SELinux (access control), and the Android Keystore (AVB key storage and FBE key wrapping).

2.3 Raspberry Pi 5 Hardware

The Raspberry Pi 5 is built around the Broadcom BCM2712 SoC featuring:

- Quad-core ARM Cortex-A76 at up to 2.4 GHz (64-bit AArch64)

- Up to 16 GB LPDDR4X RAM
- PCIe 2.0 x1 (exposed via the FFC connector)
- Dual 4K HDMI, USB 3.0, Gigabit Ethernet
- 40-pin GPIO header with UART on GPIO 14 (TX) and GPIO 15 (RX) [6]
- A dedicated 3-pin debug serial connector (between the HDMI ports)

The RPi5 firmware (VideoCore firmware) performs early hardware initialisation and hands off to the configured bootloader by reading `config.txt` from the FAT32 boot partition. By default it boots the Linux kernel Image directly, but it can be redirected to load `u-boot.bin` instead [7].

One key difference from earlier RPi boards is that the primary UART (`serial10`) is exposed via an AXI-connected PL011 UART instance, declared in the firmware Device Tree as `compatible = "arm,pl011-axi"` rather than the standard `"arm,pl011"` string [8].

2.4 U-Boot Bootloader

U-Boot is a widely-used open-source bootloader for embedded systems. It provides a scriptable boot environment supporting multiple storage media (MMC, NVMe, USB, network TFTP) and is cross-compiled for AArch64 using [9]:

```
1 CROSS_COMPILE=aarch64-linux-gnu- make rpi_arm64_defconfig
2 CROSS_COMPILE=aarch64-linux-gnu- make -j$(nproc)
```

U-Boot uses a Driver Model (DM) architecture where peripheral drivers declare compatible strings in a `udevice_id` table [10]. The boot sequence is controlled by a compiled script (`boot.scr`) produced by `mkimage`. Initial RPi5 support landed in U-Boot v2024.04; however, several limitations remained, particularly around PCIe, USB, and Ethernet initialisation [11, 12].

2.4.1 U-Boot AVB Integration

U-Boot's AVB support (`CONFIG_AVB_VERIFY`, `CONFIG_CMD_AVB`, `CONFIG_LIBAVB`) allows it to invoke the `avb verify` command to check partition signatures before booting. The public key used for verification is compiled directly into `common/avb_verify.c` as a C array (`avb_root_pub[]`).

2.5 Android Verified Boot 2.0 (AVB)

Android Verified Boot 2.0 (AVB2) is Google’s cryptographic boot-time integrity framework, defined as “the process of assuring the end user of the integrity of the software running on a device” [4]. The central data structure is the *VBMeta image*: a signed meta-data partition whose descriptors span all verified partitions, establishing a *chain of trust* rooted in the public key compiled into the bootloader.

2.5.1 VBMeta Descriptors

The VBMeta structure is cryptographically signed (SHA-256 / RSA-4096) and contains one or more typed descriptors [4]:

Hash descriptor Records the SHA-256 hash of an entire partition image (ex: `boot`). Verification confirms the partition has not been modified since signing.

Hashtree descriptor Used for large ext4 partitions (`system`, `vendor`). A 4 KiB-block Merkle tree (SHA-256) is appended to the partition image; the descriptor stores the root hash, salt value, and hashtree offset. At runtime, `dm-verity` enforces block-level integrity by recomputing hashes on each read [13].

Kernel commandline descriptor Carries verified kernel parameters that are passed to the kernel by the bootloader after successful verification. This allows security-relevant cmdline flags (ex: `androidboot.verifiedbootstate`) to be integrity-protected.

Chained partition descriptor Delegates verification authority to a separate VBMeta image residing on another partition, together with the trusted public key for that image. This enables independently signed partitions to participate in the chain of trust without embedding their hashes directly in the top-level VBMeta.

Figure 2.2 illustrates a typical AVB configuration where the top-level VBMeta partition holds hash and hashtree descriptors for `boot`, `system`, and `vendor`.

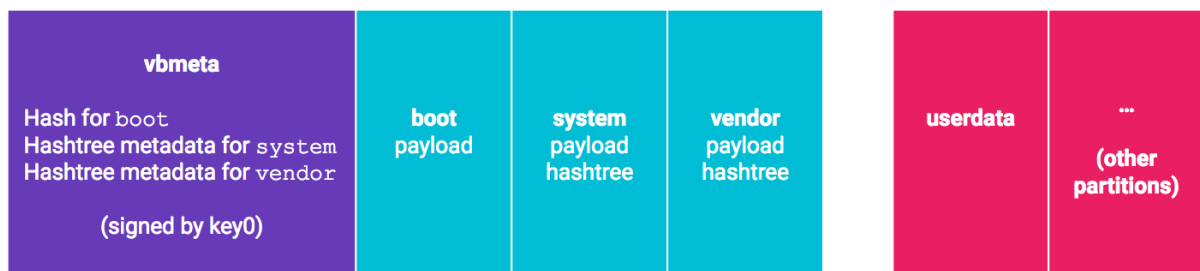


Figure 2.2: AVB with `boot`, `system`, and `vendor` partitions. The `vbmeta` partition holds signed descriptors for all three. Source: [4].

Figure 2.3 shows chained partition delegation, where the top-level VBMeta delegates authority to a per-partition VBMeta image via a public key embedded in the chained descriptor.

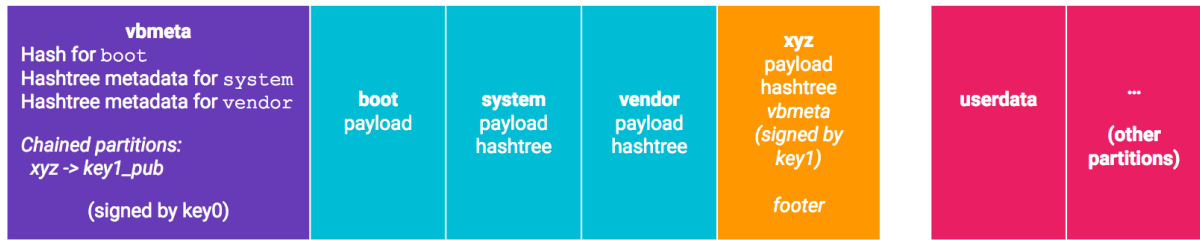


Figure 2.3: AVB with a chained partition: the top-level **vbmeta** embeds the public key used to verify an independently signed partition’s own VBMeta image. Source: [4].

2.5.2 Rollback Protection

AVB maintains *rollback indices*: monotonically increasing integers stored in tamper-evident persistent storage on the device. Each signed image carries a minimum required rollback index value; the bootloader rejects any image whose rollback index is below the stored value, preventing downgrade attacks that would re-expose patched vulnerabilities [4].

Figures 2.4 and 2.5 illustrate how rollback index values are embedded in partitions and compared against the values stored on the device.

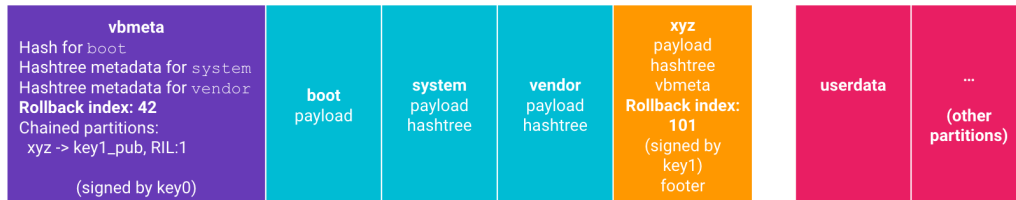


Figure 2.4: AVB rollback index values embedded in **vbmeta** and partition images. Source: [4].

2.5.3 Device States and Boot Colour Model

AVB defines two device modes and a four-colour boot state model [4]:

Locked mode Full verification is enforced. The bootloader verifies the VBMeta signature against its compiled-in root public key before loading any partition.

Unlocked mode Verification may be bypassed (ex: for development), but the device advertises an **orange** boot state to the Android framework.

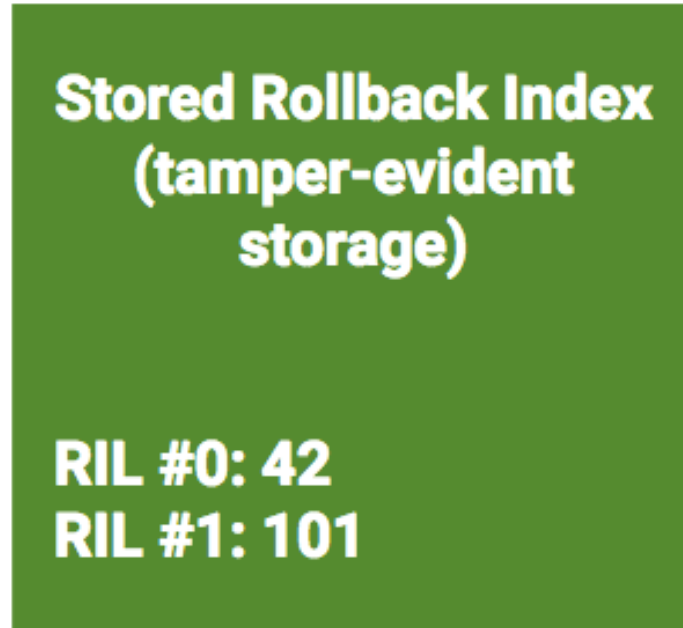


Figure 2.5: Device-stored rollback index locations. Values must be $\geq$ those in the signed metadata for boot to proceed. Source: [4].

The four boot colour states reported via `androidboot.verifiedbootstate` are: **green** (full verification passed with OEM key), **yellow** (verification passed with a non-OEM key), **orange** (unlocked / bypass mode), and **red** (verification failed or image corrupted) [4].

2.5.4 A/B Slot Support

AVB is designed to work with A/B partitioned devices, where two copies of each updatable partition are maintained (`_a` and `_b` slots). This enables seamless over-the-air updates: the update writes to the inactive slot, which is then verified and activated on the next boot, while the active slot continues to serve the running system. The slot selection metadata is maintained outside the signed VBMeta, allowing the bootloader to switch slots without re-signing [4].

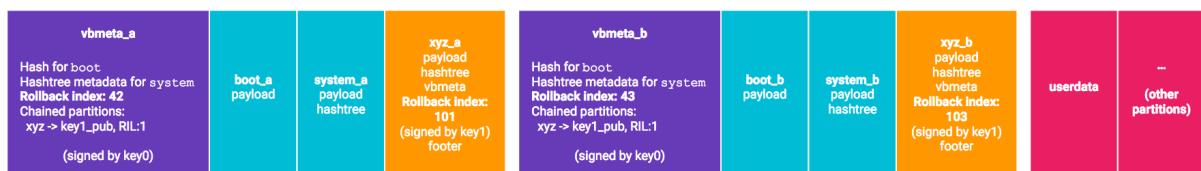


Figure 2.6: AVB with A/B partitions: both `_a` and `_b` slot images are independently signed; the bootloader selects and verifies the active slot. Source: [4].

2.5.5 libavb and avbtool

The `platform/external/avb` repository provides two key components [4, 14]:

libavb A portable C library implementing the full AVB verification logic. It requires only a small set of I/O and crypto operations to be provided by the platform, making it suitable for integration into bootloaders such as U-Boot.

avbtool A Python 3 command-line utility used at build time to generate hash and hashtree footers and to produce the final `vbmeta.img`. Key operations include `add_hash_footer`, `add_hashtree_footer`, and `make_vbmeta_image`.

The *VBMeta digest* — a cryptographic hash of the complete VBMeta partition contents — is computed by `avbtool` and can be stored by the bootloader to detect any subsequent tampering of the VBMeta partition before the kernel is loaded [4].

Figure 2.7 shows the recommended AVB boot flow, from bootloader initialisation through VBMeta verification to kernel handoff.

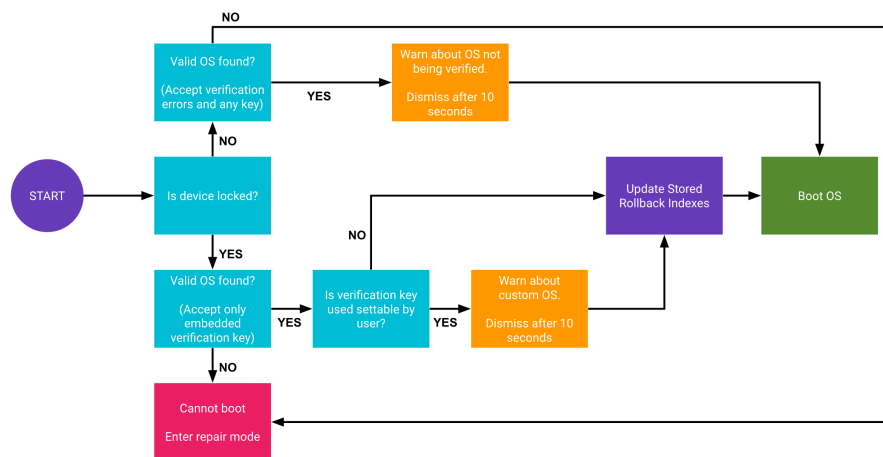


Figure 2.7: Recommended AVB boot flow: the bootloader verifies the VBMeta signature, checks rollback indices, validates partition hashes/hashtrees, and passes the verified boot state to the kernel via the commandline. Source: [4].

2.6 dm-verity

`dm-verity` is a Linux device-mapper target that enforces read-time block integrity using a Merkle hash tree [13]. On Android, it is the runtime enforcement mechanism for the AVB hashtree. For every 4 KiB block read from a verified partition, the kernel computes its SHA-256 hash and compares it against the precomputed tree. Any mismatch triggers an I/O error or, in `restart_on_corruption` mode, reboots the device.

Android 7.0 and later enhanced `dm-verity` with forward error correction (FEC) using Reed-Solomon codes, reducing the likelihood of a single corrupt sector causing a boot failure.

2.7 Full-Disk Encryption (FDE)

Full-Disk Encryption (FDE) is Android’s legacy block-level encryption mechanism, introduced in Android 5.0 [15]. It encrypts the entire `/data` partition as a single block device using the Linux `dm-crypt` device-mapper target with 128-bit AES in CBC mode and ESSIV:SHA256. Devices launched on Android 10 or later are required to use FBE instead; FDE is completely removed in Android 13. Android 10–12 retain FDE support only for devices upgrading from Android 9 or earlier.

2.7.1 Key Derivation Chain

FDE protects the Disk Encryption Key (DEK) through a seven-step derivation chain that binds decryption to both the user’s credential and a hardware key stored in the TEE [15]:

1. Generate a random 16-byte DEK and a random 16-byte salt.
2. Apply `script` to the user password and the salt, producing a 32-byte intermediate key IK1.
3. Pad IK1 with zeros to the size of the TEE-stored private key.
4. Sign the padded value with the TEE hardware key, yielding IK2.
5. Apply `script` a second time to IK2 and the salt, producing a 32-byte IK3.
6. Split IK3: the first 16 bytes become the Key Encryption Key (KEK); the last 16 bytes become the initialisation vector (IV).
7. Encrypt the DEK using AES-CBC with KEK and IV, and store the result on the device.

This chain ensures that recovering the DEK requires both the user’s credential *and* access to the TEE hardware key, preventing offline brute-force attacks on the encrypted partition.

2.7.2 Boot Flow and Service Classes

Because the entire `/data` partition is encrypted as a single unit, decryption must complete before any service that reads user data can start. Android manages this through three `init` service classes [15]:

core Services that must never stop, started immediately at boot.

main Services that restart after the user provides their credential and decryption completes.

late_start Services that begin only after `/data` has been decrypted and remounted.

For default (no-password) encryption, `vold` detects the encrypted partition, creates a `dm-crypt` mapping using the stored DEK, and mounts the decrypted filesystem directly. For password-protected devices, a temporary `tmpfs` filesystem is mounted first to display the unlock UI; after credential verification, services in the `main` and `late_start` classes are stopped, the real partition is decrypted and remounted, and those services restart. Coordination between `vold` and `init` uses properties: `vold.decrypt`, `vold.encrypt_progress`, and `ro.crypto.state`.

2.7.3 Limitations and Deprecation Rationale

FDE has two fundamental limitations that motivated its replacement by FBE [15]:

1. **No Direct Boot support.** All files in `/data` share a single DEK. The OS cannot access *any* user data — including alarms, phone calls, or accessibility services — until the user authenticates after reboot. FBE’s per-file keys enable Device-Encrypted (DE) files to be available immediately.
2. **Unrecoverable partial encryption.** A power failure during the initial in-place encryption pass leaves `/data` in a partially encrypted state with no recovery path other than a factory reset.

2.8 File-Based Encryption (FBE)

Android’s File-Based Encryption (FBE), introduced in Android 7.0 and mandatory from Android 10, encrypts each file independently using the Linux `fsencrypt` subsystem [16]. It defines two encryption classes:

Device Encrypted (DE) Available immediately after boot, before user authentication.

Used for files required during Direct Boot.

Credential Encrypted (CE) Locked until the user provides credentials (PIN, password, or biometric).

Encryption is configured via `fstab` flags on the `userdata` partition: `fileencryption=aes-256-xts:aes-256-cts:v2`. A dedicated `/metadata` partition stores the key wrapping material managed by `vold` using the `keydirectory=` flag [16, 17].

2.9 SELinux on Android

Android enforces Mandatory Access Control (MAC) through SELinux. From Android 8.0 onward, platform and vendor policies are built and versioned independently [18]. Device bring-up typically begins in *permissive* mode (denials logged but not enforced), transitioning to *enforcing* mode as policy matures [19]. Policy is precompiled at build time into `/vendor/etc/selinux/precompiled_sepolicy`; at boot, `init` validates it against SHA-256 hashes before loading.

2.10 Android Init System

Android’s `init` is the first userspace process. It parses `.rc` files (Android Init Language) to manage services and trigger actions on property changes. First-stage `init` is responsible for mounting `/system` and `/vendor` from the `fstab`; at this point it also runs the AVB verification policy from the `fstab avb=` flags. Second-stage `init` handles service lifecycle, SELinux policy loading, and property management.

2.11 ISO/SAE 21434 and Threat Analysis and Risk Assessment (TARA)

ISO/SAE 21434 [5] is the cybersecurity engineering standard for road vehicles. Its Threat Analysis and Risk Assessment (TARA) process (Clause 15) provides a structured methodology for identifying, evaluating, and treating cybersecurity risks in automotive systems. Although this project targets a research platform rather than a production automotive ECU, the TARA methodology is applied to characterise the security posture and to identify the gap between the current implementation and production-grade hardening requirements.

2.11.1 Asset Identification

An *asset* is any component whose loss of confidentiality, integrity, or availability would harm the security posture or operational function of the system. Assets are categorised by type (credential, image, configuration, interface, build artifact, data) and annotated with the security properties at risk: Confidentiality (C), Integrity (I), and Availability (A).

2.11.2 STRIDE Threat Modelling

Threats are enumerated by applying the STRIDE model [20] to each asset:

Spoofing An entity falsely claims to be another (ex: a fake AVB public key).

Tampering Modification of data or code without authorisation (ex: altering a partition image on the SD card).

Repudiation Denying that an action took place (ex: absence of a build audit trail).

Information Disclosure Unauthorised access to data (ex: reading a plaintext /data partition from a removed SD card).

Denial of Service Making the system unavailable (ex: corrupting `vbmeta` under a fail-closed policy).

Elevation of Privilege Gaining higher access than intended (ex: gaining an interactive U-Boot shell via UART).

2.11.3 Risk Scoring

Impact is assessed using four SFOP damage categories [5] on a 1–4 scale. The *highest single category score* is used as the overall impact.

Table 2.2: SFOP Impact Scale (ISO/SAE 21434 Clause 15)

Score	Label	Safety (S)	Financial (F)	Operational (O)	Privacy (P)
1	Negligible	No effect	Negligible cost	Minor inconvenience	No personal data
2	Moderate	Indirect risk	Moderate cost	Degraded function	Limited PII
3	Major	Direct (non-life)	Significant cost	Key function loss	Significant PII
4	Severe	Life-threatening	Severe damage	Complete loss	Mass data breach

Feasibility is scored 1–4 based on attack potential attributes: elapsed time, expertise, knowledge of the target, window of opportunity, and equipment required.

Risk Level = Impact $\times$ Feasibility, classified as: Low (1–4), Medium (5–8), High (9–12), Critical (13–16).

2.11.4 Risk Treatment and Residual Risk

Each risk receives a treatment decision: (1) avoid — eliminate the asset or scenario, (2) reduce — apply a countermeasure, (3) share — transfer responsibility, or (4) accept — document the risk explicitly with a rationale. Risks that cannot be fully mitigated are recorded as *residual risks* and must be accepted with a named owner and follow-up action. ISO/SAE 21434 prohibits silently omitting unmitigated risks.

Chapter 3

SYSTEM ARCHITECTURE

3.1 Overview

The `meta-rpi5-secure-aosp` project delivers a complete, reproducible pipeline for building and deploying a security-hardened Android Automotive OS image onto a Raspberry Pi 5. The system has two primary concerns: a *build-time pipeline* that produces a signed, encrypted SD card image, and a *runtime boot chain* that verifies and mounts partitions before handing off to Android userspace.

3.2 Boot Chain

The secure boot chain consists of five layers, each responsible for initializing and handing off to the next:

1. **RPi Firmware** (VideoCore, closed-source): Powers on, initializes DRAM and PCIe, reads `config.txt` from the FAT32 boot partition, and loads `u-boot.bin`. The firmware also provides the Device Tree blob to U-Boot via the ARM64 boot protocol [7].
2. **U-Boot**: Runs the compiled boot script (`boot_avb.scr`). Invokes `avb init` and `avb verify` to check the VBMeta structure against the compiled-in root public key. On success, loads the Android kernel `Image` and `ramdisk.img` from the FAT32 partition and calls `booti` [9,11].
3. **Android Kernel** (Linux 6.12, `android_rpi5_defconfig`): Decompresses and initializes hardware. Diagnostic markers (RPi5-DIAG:) are emitted via `pr_notice()` at four boundaries: `start_kernel` entry, after `console_init`, at `do_initcalls`, and before `run_init_process`.

4. **Android Init (first-stage)**: Mounts `/system` and `/vendor` using the `fstab` embedded in the ramdisk. The `avb=vbmeta` flag triggers dm-verity verification of both partitions via the `vbmeta` partition [13].
5. **Android Init (second-stage) and userspace**: Loads SELinux policy, starts `vold` (which activates FBE on `/data`), and launches the Android Automotive framework [16].

Figure 3.1 illustrates the complete boot flow across the five coloured layers described above. Each layer either advances control along the green *Yes* path or routes to a device reboot along the red *No* path. The three diamond-shaped decision nodes represent the cryptographic verification gates: the AVB signature check performed by U-Boot against its compiled-in root public key; the dm-verity block-integrity check applied to `/system` and `/vendor` at kernel mount time; and the FBE key-derivation check performed by `vold` before `/data` becomes accessible. All three failure paths converge on a single *Boot Failure / Reboot* terminal, enforcing a fail-closed policy with no degraded-mode fallback.

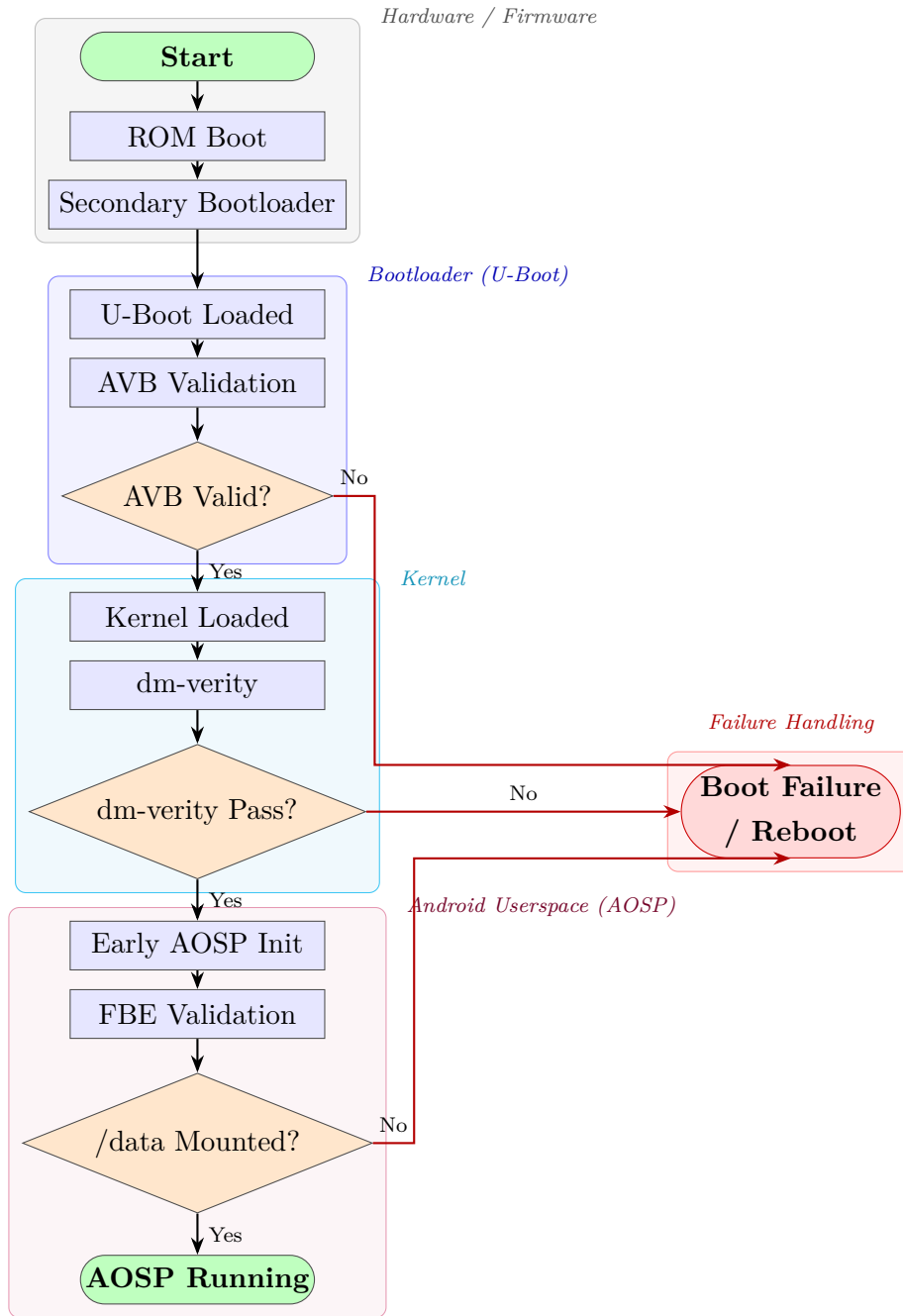


Figure 3.1: Secure boot chain on Raspberry Pi 5. Three verification gates (AVB, dm-verity, FBE) each enforce a fail-closed policy: any verification failure triggers a device reboot with no fallback to a degraded boot state.

3.3 Partition Layout

The GPT-partitioned SD card is defined in `config/sdcard/uboot_aosp_signed.yaml`. The Table 3.1 describes the layout.

The non-contiguous partition numbers (p1, p3, p5–p8) are intentional: p2 and p4 are reserved for alternative configurations without breaking existing `fstab` or U-Boot environment references.

Table 3.1: GPT Partition Layout (signed build, 14.6 GB SD card)

No.	Name	Format	Size (MB)	Contents
p1	boot	FAT32	128	RPi firmware, U-Boot, kernel, ramdisk, <code>boot.scr</code>
p3	userdata	ext4	(fill)	<code>/data</code> — FBE-encrypted user data
p5	system	ext4	3072	<code>/system</code> — AVB hashtree signed
p6	vendor	ext4	384	<code>/vendor</code> — AVB hashtree signed
p7	metadata	ext4	16	<code>/metadata</code> — FBE key material
p8	vbmata	raw	4	AVB top-level VBMata image

3.4 Build Pipeline

The build pipeline is orchestrated by `rpi5-build`, a Python CLI implemented in `src/meta_rpi5_secure`. It is driven by a YAML configuration file (`config/rpi5_uboot_aosp_signed.yaml`) and accepts per-run overrides via CLI flags.

3.4.1 Stages

sync Clones U-Boot at a pinned commit, initializes the AOSP repo with the `android-16.0.0_r4` manifest, and applies local manifest overlays from `ganeshhalthota/android_local_manifest` [21]. Optionally clones the kernel source from `raspberrypi-vanilla/android_kernel_brcm_rpi` at a pinned commit.

patch Applies `git format-patch` stacks from `patches/uboot/`, `patches/kernel/`, and `patches/aosp/device_brcm_rpi5/` using `git am -3way`. Idempotent: already-applied patches are detected via `git log -grep` on the commit subject, preventing double-application.

build Compiles U-Boot (`rpi_arm64_defconfig`), enables AVB Kconfig symbols (`CONFIG_LIBAVB`, `CONFIG_AVB_VERIFY`, `CONFIG_CMD_AVB`), injects the AVB root public key into `avb_verify.c`, compiles the boot script (`boot_avb.scr`), and builds AOSP (`bootimage`, `systemimage`, `vendorimage`).

sign Signs `system.img` and `vendor.img` with AVB hashtree footers (SHA-256/RSA-4096), shrinks images if needed to fit within the predefined partition sizes, and generates the combined `vbmata.img` [4].

sdcard Assembles the signed images and supporting files into a flashable GPT image using the partition layout YAML.

3.4.2 Configuration Model

The build is governed by a two-layer configuration model:

- **rpi5 config YAML** (`config/rpi5_uboot_aosp_signed.yaml`): Controls source tags, AVB key paths, signing behaviour, SELinux mode, encryption mode, cmdline profile, and boot state.
- **sdcard config YAML** (`config/sdcard/uboot_aosp_signed.yaml`): Defines the GPT partition layout, sizes, image paths, and extra file mappings.

A `BuildContext` dataclass carries the resolved configuration across all stage modules, ensuring a single source of truth for all options. Security-sensitive options (`boot_state_override = orange`, `avb_fail_policy = fail_open`) are blocked by default and require the explicit `-allow-insecure-boot-state` flag.

3.5 Security Design Decisions

3.5.1 External Signing Pipeline

The AVB signing is deliberately separated from the AOSP build (`BOARD_AVB_ENABLE = false`). This design:

- Keeps the private key outside the AOSP build environment.
- Allows the same AOSP images to be signed with different keys (development vs. production) without rebuilding.
- Matches the typical automotive production workflow where signing is a separate, audited step.

3.5.2 Fail-Closed AVB Policy

The default `avb_fail_policy = fail_closed` causes U-Boot to call `reset` if AVB initialization or verification fails. This prevents a compromised image from booting in a degraded state and is the recommended policy for production devices [4].

3.5.3 FBE Dependency Chain

FBE requires a strict prerequisite chain enforced by the build tool at startup:

1. `sdcard.enable_signing = true` (AVB must be active).
2. `avb.uboot_fail_policy = fail_closed`.
3. `userdata` and `metadata` partitions present in the sdcard config.

If any prerequisite is missing, the build tool raises a descriptive error before any work is performed, preventing a partially-configured image from being produced.

Chapter 4

IMPLEMENTATION

This chapter describes the implementation of each component in detail, organized by the build pipeline stage in which it is introduced.

4.1 U-Boot Patch: PL011-AXI Serial Driver

4.1.1 Problem

The RPi5 firmware Device Tree declares `serial10` (the default `stdout-path` target) with `compatible = "arm,pl011-axi"` [22]. U-Boot's PL011 driver (`drivers/serial/serial_pl01x.c`) did not include this string in its `udevice_id` match table [10]. As a result, U-Boot couldn't bind the UART node and produced no serial output, making all subsequent debugging impossible.

4.1.2 Solution

Patch 0001-serial-pl01x-add-arm-pl011-axi-compatible-for-RPi5.patch adds the "arm,pl011-axi" entry to `pl01x_serial_id[]` with `TYPE_PL011`:

Listing 4.1: in `serial_pl01x.c`] Addition to `pl01x_serial_id[]` in `serial_pl01x.c`

```
1 /* RPi5 firmware DT uses "arm,pl011-axi" for serial10 (stdout-path). */
2 {.compatible = "arm,pl011-axi", .data = TYPE_PL011},
3 {.compatible = "arm,pl011",      .data = TYPE_PL011},
4 {.compatible = "arm,pl010",      .data = TYPE_PL010},
```

The ARM PL011 TRM (DDI0183G) defines the AXI variant as register-compatible with the standard PL011 [8], making the `TYPE_PL011` assignment correct.

4.1.3 Risk Assessment

Low risk. The change only adds a new entry to the match table; existing "arm,pl011" bindings are unaffected. Boards that do not use "arm,pl011-axi" in their Device Tree will not be matched by this entry.

4.2 U-Boot AVB Kconfig Configuration

Enabling U-Boot AVB requires a specific set of Kconfig symbols. The build stage automates this via `scripts/config`:

Listing 4.2: Kconfig symbols enabled for AVB

```

1 ./scripts/config --enable CONFIG_ANDROID_BOOT_IMAGE
2 ./scripts/config --enable CONFIG_UDP_FUNCTION_FASTBOOT # selects hidden
  CONFIG_FASTBOOT
3 ./scripts/config --enable CONFIG_LIBAVB
4 ./scripts/config --enable CONFIG_AVB_VERIFY
5 ./scripts/config --enable CONFIG_CMD_AVB
6 ./scripts/config --set-val CONFIG_FASTBOOT_BUF_ADDR 0x10000000
7 ./scripts/config --set-val CONFIG_AVB_BUF_ADDR 0x10000000

```

The buffer address `0x10000000` (256 MiB) is set explicitly because neither symbol has a default in the upstream `rpi_arm64_defconfig`. The value is chosen to sit above all standard ARM64 boot-image load regions defined in `board/raspberrypi/rpi/rpi.env`:

Table 4.1: RPi5 ARM64 U-Boot load addresses vs. AVB buffer placement

Region	Symbol / use	Address
Kernel Image	<code>kernel_addr_r</code>	<code>0x00080000</code>
Decompression area	<code>kernel_comp_addr_r</code>	<code>0x02000000</code>
Device Tree Blob	<code>fdt_addr_r</code>	<code>0x05600000</code>
Ramdisk	<code>ramdisk_addr_r</code>	<code>0x05700000</code>
Ramdisk end (est.)	~50 MiB max	~ <code>0x08900000</code>
AVB / Fastboot buffer	<code>CONFIG_AVB_BUF_ADDR</code>	<code>0x10000000</code>

The 119 MiB gap between the estimated ramdisk end and the buffer start ensures no overlap under any realistic ramdisk size. The 112 MiB buffer (`CONFIG_AVB_BUF_SIZE = 0x7000000`) is the U-Boot default and is sufficient to hold any Android partition image presented to the AVB verification command.

After setting these symbols, `make olddefconfig` resolves the full dependency graph. The build stage then verifies that all required symbols are present in `.config` before proceeding; missing symbols raise a descriptive error.

4.2.1 Public Key Injection

The AVB root public key (`avb_public_key.bin`, in AVB packed format) is read and embedded as a C array in `common/avb_verify.c`. The build stage uses a regular expression to locate and replace the existing `avb_root_pub[]` array in-place, making the operation idempotent. A warning is printed if signing is enabled but U-Boot isn't being rebuilt, reminding the operator to rebuild U-Boot after any key rotation.

4.3 U-Boot Boot Script

The AVB boot script (`config/uboot/boot_avb.cmd`) implements the following sequence:

1. Resolve `avb_fail_policy` (default injected at build time via the template placeholder `__AVB_FAIL_POLICY__`).
2. `avb init 0` — attach the AVB device on MMC 0.
3. `avb verify` — verify all descriptors in `vbmata.img`. On failure: either reset (`fail_closed`) or continue with orange boot state (`fail_open`).
4. `fatload mmc 0:1` — load kernel Image and `ramdisk.img`.
5. Construct `bootargs`: append `avb_bootargs` (populated by `avb verify`), SELinux mode, encryption mode, and optional cmdline profile.
6. `booti` — hand off to the Android kernel.

The script is compiled to binary by `mkimage`:

```
1 mkimage -C none -A arm64 -T script -d boot_avb.cmd boot_avb.scr
```

Template placeholders (`__SELINUX_MODE__`, `__ENCRYPTION_ARGS__`, `__CMDLINE_PROFILE_ARGS__`) are resolved at build time before `mkimage` is invoked, keeping the configuration model in the YAML layer rather than hardcoded in the script.

4.4 Kernel Patch: Boot-Layer Diagnostic Markers

To measure and verify the progression through the kernel boot sequence, four `pr_notice()` markers are injected into `init/main.c`:

`pr_notice` (priority 5) is visible at the default kernel loglevel and guaranteed to be present in the serial console output. The `RPI5-DIAG:` prefix makes the markers `grep`-able in captured logs.

Table 4.2: Kernel boot-layer diagnostic markers

Layer	Location in init/main.c	Message (prefix: RPi5-DIAG)
1	Entry of <code>start_kernel()</code>	Layer 1 <code>start_kernel</code> entry
2	After <code>console_init()</code>	Layer 2 <code>console_init</code> complete
3	Before <code>do_initcalls()</code>	Layer 3 <code>do_initcalls</code> entry
4	Before <code>run_init_process()</code>	Layer 4 <code>kernel_init</code> userspace exec

4.5 AOSP Patch: AVB-Aware fstab Selection

The upstream RPi5 device tree does not provide an AVB-aware `fstab`.

Patch 0001-rpi5-add-avb-aware-fstab-selection.patch adds:

- `RPI5_ENABLE_AVB` build toggle in `BoardConfig.mk`.
- `androidboot.vbmeta.device=/dev/block/by-name/vbmeta` kernel command line argument, so first-stage init can locate the VBMeta partition by GPT label.
- `ramdisk/fstab.rpi5.avb`: AVB-aware `fstab` with `avb=vbmeta` on both `/system` and `/vendor`.
- Conditional selection in `device.mk`: when `RPI5_ENABLE_AVB=true`, the AVB `fstab` is used in both `ramdisk` and `vendor` partitions.

Listing 4.3: `fstab.rpi5.avb` — system and vendor entries

```

1 /dev/block/by-name/system /system ext4 ro,barrier=1
2   wait,avb=vbmeta,first_stage_mount
3 /dev/block/by-name/vendor /vendor ext4 ro,barrier=1
4   wait,avb=vbmeta,first_stage_mount

```

The `avb=vbmeta` flag instructs first-stage init to look up the VBMeta descriptor for each partition before mounting it.

4.6 AOSP Patch: SELinux Enforcing Mode

The upstream RPi5 AOSP device tree ships SELinux in Permissive mode, where policy violations are logged but not enforced. The objective was to enable SELinux Enforcing mode and confirm that the platform’s base policy is compatible with RPi5 bring-up.

4.6.1 Policy Structure

Android precompiles the SELinux policy at build time into `/vendor/etc/selinux/precompiled_sepolicy`. The policy is a binary merge of the platform policy

(PLAT_SEPOLICY), vendor policy (BOARD_SEPOLICY), and their mapping files. At boot, `init` validates the precompiled policy against SHA-256 hashes stored in `plat_sepolicy_and_mapping.sha256` before loading it into the kernel via `security_load_policy()` [18].

The RPi5 device tree does not define device-specific policy modules. It inherits the base AOSP platform policy unchanged, which is sufficient for the subsystems under evaluation (AVB, FBE, boot diagnostics). No custom `.te` policy files or file-context entries were added beyond those introduced by the `e2fsdroid` fix described in Section 4.8.

4.6.2 Android 16 Enforcing Behaviour

In Android 16 (`android-16.0.0_r4`), SELinux transitions to Enforcing mode unconditionally at policy load time. The `androidboot.selinux=permissive` kernel argument, honoured in earlier Android releases, is ignored under this build configuration. `init`'s `IsEnforcing()` calls `security_getenforce()` after loading the precompiled policy and does not re-read the kernel command line permissive flag. There is therefore no safe run-time fallback to Permissive mode: any policy violation preventing a required service from starting will cause a boot failure.

4.6.3 Configuration

SELinux mode is controlled by the `boot.selinux_mode` key in the `rpi5` build config YAML (e.g. `config/rpi5_uboot_aosp_signed.yaml`):

Listing 4.4: `boot.selinux_mode` in `rpi5_uboot_aosp_signed.yaml`

```
1 boot:
2   # Options: permissive | enforcing
3   selinux_mode: permissive    # set to enforcing for production
```

During the build stage, `stages/build.py` renders the `__SELINUX_MODE__` placeholder in the boot script template (`config/uboot/boot_avb.cmd`) with this value:

Listing 4.5: `androidboot.selinux` injected into bootargs in `boot_avb.cmd`

```
1 setenv bootargs "${bootargs}..._androidboot.selinux=__SELINUX_MODE__
..."
```

The mode can also be overridden per-run via the `-selinux-mode` CLI flag without modifying the YAML. To enable SELinux Enforcing mode, set `boot.selinux_mode: enforcing` in the config YAML before running the build stage.

4.6.4 AVC Denial Handling

During bring-up, SELinux enforcement failures appear as AVC (Access Vector Cache) denial messages in both `dmesg` and `logcat`:

```

1  avc: denied { <permission> } for comm="<process>"
2      scontext=<source_context> tcontext=<target_context>
3      tclass=<object_class> permissive=0

```

The `permissive=0` field confirms the denial was enforced rather than just logged. The `rpi5_boot_diag.sh` service captures the SELinux enforcement mode (`ro.boot.selinux`, `getenforce`) as part of its post-boot state report, providing per-boot confirmation of the enforcement posture.

4.6.5 Serial Log Evidence

Non-Secure Device (`selinux_mode: permissive`)

The signed build with `selinux_mode: permissive` produced the following kernel command line and property state:

Listing 4.6: Kernel command line — permissive build

```

1  Kernel command line: stack_depot_disable=on cgroup_disable=pressure
2      root=/dev/ram0 rootwait androidboot.hardware=rpi5
3      androidboot.selinux=permissive ignore_loglevel loglevel=7
4      initcall_debug androidboot.verifiedbootstate=green
5      androidboot.vbmeta.device_state=locked
6      androidboot.serialno=6113ca6606956aab

```

Listing 4.7: Selected getprop output — permissive build

```

1  [ro.boot.selinux]:          [permissive]
2  [ro.boot.verifiedbootstate]: [green]
3  [ro.crypto.state]:         [unsupported]

```

Secure Device (`selinux_mode: enforcing`)

The signed build with `selinux_mode: enforcing` produced:

Listing 4.8: Kernel command line — enforcing build

```

1  Kernel command line: stack_depot_disable=on cgroup_disable=pressure
2      root=/dev/ram0 rootwait androidboot.hardware=rpi5
3      androidboot.selinux=enforcing ignore_loglevel loglevel=7
4      initcall_debug androidboot.verifiedbootstate=orange
5      androidboot.vbmeta.device_state=unlocked
6      androidboot.serialno=6113ca6606956aab

```

Listing 4.9: Selected getprop output — enforcing build

```

1  [ro.boot.selinux]:          [enforcing]
2  [ro.boot.verifiedbootstate]: [orange]

```



```
3 [ro.crypto.state]: [unsupported]
```

Listing 4.10: Policy transition and first AVC denials — enforcing build

```
1 [23.245] audit: enforcing=1 old_enforcing=0 lsm=selinux
2 [23.407] avc: denied { set } for
3         property=ro.boot.audio.tinyalsa.simulate_input
4         scontext=u:r:vendor_init:s0
5         tcontext=u:object_r:bootloader_prop:s0
6         tclass=property_service permissive=0
7 [23.451] avc: denied { set } for
8         property=persist.bluetooth.a2dp_aac.vbr_supported
9         scontext=u:r:vendor_init:s0
10        tcontext=u:object_r:bluetooth_prop:s0
11        tclass=property_service permissive=0
12 [23.495] avc: denied { set } for
13        property=ro.lockscreen.disable.default
14        scontext=u:r:vendor_init:s0
15        tcontext=u:object_r:default_prop:s0
16        tclass=property_service permissive=0
17 [23.683] avc: denied { search } for comm="ueventd" name="/"
18        dev="mmcblk0p7" scontext=u:r:ueventd:s0
19        tcontext=u:object_r:unlabeled:s0
20        tclass=dir permissive=0
```

The `permissive=0` field in every entry confirms enforcement was active. The denials are RPi5-specific policy gaps (vendor properties, unlabelled `/metadata` root inode) that require device-specific policy extensions for a production deployment.

4.7 AOSP Patch: File-Based Encryption

Patch 0002-rpi5-add-fbe-file-based-encryption.patch enables FBE on userdata [16]:

- `RPI5_ENABLE_FBE` toggle extends the three-way fstab selection: `FBE > AVB-only > base`.
- `ramdisk/fstab.rpi5.fbe`: extends the AVB fstab with FBE flags on `/data` and a `/metadata` entry.
- `ro.crypto.type=file` product property signals the crypto stack.
- `androidboot.encryption_mode=fbe` is appended to the kernel command line.

The critical `fstab` entry for userdata:

Listing 4.11: fstab.rpi5.fbe — userdata entry (FBE + metadata encryption)

```

1 /dev/block/by-name/userdata /data ext4
2     nodev,noatime,nosuid,errors=panic
3     wait,check,formattable,quota,
4     fileencryption=aes-256-xts:aes-256-cts:v2,
5     keydirectory=/metadata/vold/metadata_encryption

```

The `fileencryption=aes-256-xts:aes-256-cts:v2` flag selects AES-256-XTS for file content and AES-256-CTS for filenames at policy version 2. The `keydirectory=` flag activates metadata encryption: `vold` stores the wrapped key in the `/metadata` partition, which is itself not encrypted, ensuring key material survives a factory reset without persisting in `/data` [17].

4.8 FBE Bring-up: Metadata Partition SELinux Boot Loop

4.8.1 Symptom

After flashing a signed SD card image with FBE enabled, the device entered a repeating reboot loop. Each cycle completed in approximately 9–10 seconds, with AVB verification passing on every cycle:

Listing 4.12: Reboot loop symptom on UART console

```

1 AVB: Verification PASSED
2 ...
3 [2.8s] vold: Failed to create /metadata/vold: Permission denied
4 [2.8s] vold: read_key failed in mountFstab
5 [9.4s] reboot: Restarting system with command 'bootloader'

```

Because AVB verification passed, the signing pipeline was confirmed correct. The failure was inside the FBE initialization path.

4.8.2 Diagnosis: Two Concurrent Root Causes

Full debug visibility was obtained by rebuilding `boot_avb.scr` with `-cmdline-profile debug (ignore_loglevel loglevel=7 initcall_debug)`, which exposed the complete init sequence on the serial console.

Root Cause A — SELinux Enforcing Despite Permissive Kernel Argument

The bootargs carried `androidboot.selinux=permissive`, but the serial log showed:

```

1 audit: enforcing=1 old_enforcing=0

```

All subsequent AVC denials showed `permissive=0` (enforced). In Android 16 (`android-16.0.0_r4`), `init`'s `IsEnforcing()` function ignores the command-line permissive flag under this build configuration — SELinux transitions to enforcing unconditionally at policy load time.

Root Cause B — Metadata Partition Root Inode is unlabeled

The SD card build stage created the `metadata` filesystem with bare `mkfs.ext4`, which produces a filesystem with no SELinux extended attributes. The partition root directory inode therefore carried the default context `u:object_r:unlabeled:s0`.

The compiled `plat_file_contexts` already contained the correct mapping:

```
1 /metadata(/.*)?      u:object_r:metadata_file:s0
2 /metadata/vold(/.*)? u:object_r:vold_metadata_file:s0
```

The label was simply never applied to the inode. When `vold` (context `u:r:vold:s0`) attempted to create `/metadata/vold/`, SELinux denied it:

```
1 avc: denied { write } for comm="binder:344_2" name="/"
2   scontext=u:r:vold:s0 tcontext=u:object_r:unlabeled:s0
3   tclass=dir permissive=0
```

Without `/metadata/vold/`, `vold` couldn't store the FBE master key, which is required by the `keydirectory= fstab` flag before `/data` can be mounted.

4.8.3 Fix: Replace `mkfs.ext4` with `e2fsdroid`

The fix replaced the bare `mkfs.ext4` call in the SD card stage with `e2fsdroid`, the AOSP-provided tool that creates ext4 filesystems with SELinux extended attributes pre-populated from the compiled `file_contexts` map:

Listing 4.13: `e2fsdroid` invocation for labelled metadata partition

```
1 e2fsdroid -e -T 0 \
2   -S ${SELINUX_FC} \   # compiled file_contexts
3   -L /metadata        \   # mount point label root
4   ${METADATA_IMAGE}
```

With `e2fsdroid`, the metadata partition root inode is created with `u:object_r:metadata_file:s0`, `vold` can create `/metadata/vold/`, and FBE initialisation succeeds.

4.8.4 Security Significance

This incident demonstrates the interdependency between the MAC layer (SELinux) and the encryption layer (FBE): a correct FBE `fstab` configuration fails silently if the partition on which the key material must be stored lacks the correct SELinux label. The root

cause was not a security vulnerability but a toolchain gap — `mkfs.ext4` does not apply SELinux labels, while `e2fsdroid` does. For a verified boot stack, where every layer must be correctly labelled for the MAC policy to permit operations, using the AOSP-provided image creation tools is mandatory.

4.9 AOSP Patch: Boot Diagnostics Service

Patch 0003-rpi5-add-boot-diagnostics.patch adds a post-boot diagnostic service:

init.rpi5.diag.rc Defines a oneshot service triggered by `onproperty:sys.boot_completed=1`. The `console` keyword routes `stdout` to `/dev/console`, making output appear on the UART without ADB.

rpi5_boot_diag.sh Collects and prints: build info, AVB verified boot state (`ro.boot.verifiedbootstate`), VBMeta digest, crypto state (`ro.crypto.state`, `ro.crypto.type`), FBE properties, vold status, fstab, mount table, `dm-*` block devices, and SELinux enforce mode. Output is simultaneously written to `/data/local/tmp/rpi5_boot_diag.log`.

This service provides an unambiguous, per-boot record of the security state of the device without requiring ADB connectivity, which is particularly useful during bring-up when network and USB may not yet be functional.

4.10 AVB Signing Stage

The sign stage (`src/meta_rpi5_secure_aosp/stages/sign.py`) performs:

1. For each of `system.img` and `vendor.img`:
 - (a) Copy to `.signed.img`.
 - (b) Calculate the maximum payload size for an AVB hashtree footer via `avbtool calculate_max_image_size`.
 - (c) If the image exceeds this size: `e2fsck -fy` → `resize2fs` → `truncate` to the target size.
 - (d) Append the AVB hashtree footer via `avbtool add_hashtree_footer` (algorithm: SHA256/RSA-4096, key: `avb_private_key.pem`).

Generate `vbmeta.img` via `avbtoolmake_vbmeta_image`, referencing the signed `system.signed.img` and `vendor.signed.img` [4, 14].

- 2.

The `AvbTool` wrapper class in `utils/avb.py` abstracts the `avbtool` invocations and resolves the tool path from the AOSP source tree.

4.11 Build Orchestrator CLI

The `rpi5-build` CLI (`src/meta_rpi5_secure_aosp/main.py`) provides:

- Stage selection: `-stage {all|sync|patch|build|sdcard}`
- Component selection: `-code {all|kernel|uboot|aosp}`
- Per-run security overrides: `-build-variant`, `-selinux-mode`, `-boot-state-override`, `-avb-fail-policy`, `-encryption-mode`
- An explicit `-allow-insecure-boot-state` guard that must be provided before any insecure option (`orange state`, `fail_open`) is accepted.

The `BuildContext` dataclass is the single shared state object threaded through all stage modules, eliminating global state and making each stage independently testable.

Chapter 5

RESULTS AND EVALUATION

5.1 Overview

This chapter presents the results obtained from executing the complete build pipeline and booting the resulting image on a Raspberry Pi 5 Model B (16 GB RAM variant). All observations were made via UART serial console at 115200 baud using `minicom`. The serial interface connects to GPIO 14 (TX) and GPIO 15 (RX) on the 40-pin header [6].

5.2 Build Pipeline Execution

The full pipeline was executed using:

```
1 rpi5-build --config config/rpi5_uboot_aosp_signed.yaml \  
2           --stage all --code all
```

5.2.1 U-Boot Build

U-Boot was compiled from commit `eefb822fb574` (pinned in the config) using `rpi_arm64_defconfig`. After AVB Kconfig injection, the `.config` contained all required symbols:

```
1 CONFIG_ANDROID_BOOT_IMAGE=y  
2 CONFIG_FASTBOOT=y  
3 CONFIG_LIBAVB=y  
4 CONFIG_AVB_VERIFY=y  
5 CONFIG_CMD_AVB=y  
6 CONFIG_FASTBOOT_BUF_ADDR=0x10000000  
7 CONFIG_AVB_BUF_ADDR=0x10000000
```

The AVB root public key (1032 bytes, SHA-256/RSA-4096 packed format) was injected into `common/avb_verify.c` and the build completed successfully.

5.2.2 AOSP Build

The AOSP tree was built with `RPI5_ENABLE_AVB=true RPI5_ENABLE_FBE=true`, producing:

- `boot.img` — Android kernel + ramdisk with `fstab.rpi5.fbe`
- `system.img` — ext4 system partition (pre-signing)
- `vendor.img` — ext4 vendor partition (pre-signing)

5.2.3 Signing Stage

The sign stage processed both partitions:

- `system.img`: resized to fit the 3072 MB AVB partition limit, hashtree footer appended.
- `vendor.img`: resized to fit the 384 MB AVB partition limit, hashtree footer appended.
- `vbmeta.img`: generated referencing both signed images.

5.2.4 SD Card Image Assembly

The signed image (`rpi5-uboot-aosp-signed-user-202606040454.img`, 14.3 GiB) was flashed to a 64 GiB SD card using `scripts/flash_sdcard.sh`. Because the image is built for a fixed partition layout smaller than the target card, the script performs a three-step post-flash expansion:

1. **Relocate backup GPT header** to the physical end of the 64 GiB disk.
2. **Expand userdata (p3)** to fill the remaining space, growing from its reserved size to 56.0 GiB.
3. **Resize the ext4 filesystem** on `/dev/sdc3` to match the new partition boundary.

The resulting partition layout confirms all six partitions from Table 3.1 are correctly placed and sized:

Listing 5.1: Final GPT partition layout after flashing to 64 GiB SD card

	Number	Start (sector)	End (sector)	Size	Name
1	1	2048	264191	128.0 MiB	boot
3	3	7383040	124735454	56.0 GiB	userdata (expanded)
4	5	264192	6555647	3.0 GiB	system
5	6	6555648	7342079	384.0 MiB	vendor

6	7	7342080	7374847	16.0 MiB	metadata
7	8	7374848	7383039	4.0 MiB	vbmeta

The `userdata` partition fills whatever space remains on the card, making the pipeline card-size agnostic: the same signed image can be flashed to any SD card of 14.3 GiB or larger without rebuilding.

5.3 Boot Sequence Validation

Boot validation was performed across two image variants: an unsigned `eng` build (no AVB, SELinux permissive, `fail_open` policy) and a signed `user` build (AVB enabled, SELinux enforcing, `fail_closed` policy). The unsigned build provides a functional baseline; the signed build validates the security stack. Negative test cases within each group confirm that the security mechanisms reject invalid inputs rather than silently accepting them.

5.3.1 Unsigned Build Validation

The unsigned build uses a plain U-Boot boot script without AVB commands, an MBR partition layout (no vbmeta partition), and `androidboot.selinux=permissive`. Five test cases validate its expected behaviour.

TC-U-001 — Unsigned image boots to Android. U-Boot loaded the kernel and ramdisk from the FAT32 boot partition and transferred control to the kernel without invoking any AVB commands:

Listing 5.2: TC-U-001: U-Boot output on unsigned image

```

1  ** Booting bootflow 'mmc@fff000.bootdev.part_1' with script
2  === Android U-Boot Boot Script ===
3  Loading kernel Image...
4  37157376 bytes read in 1577 ms (22.5 MiB/s)
5  Loading ramdisk...
6  1544522 bytes read in 68 ms (21.7 MiB/s)
7  Booting Android...
```

Android completed initialisation and the serial console became responsive, confirming a full unsigned boot.

TC-U-002 — Boot partition contains correct files for unsigned image. The FAT32 boot partition (p1) was inspected on the host before flashing. The plain (non-AVB) boot script was present and `boot_avb.scr` was absent, confirming the unsigned pipeline produced the correct boot artefacts:

Listing 5.3: TC-U-002: Boot partition file listing — unsigned image

```

1 -rwxr-xr-x boot.scr      1,499  (non-AVB variant)
2 -rwxr-xr-x cmdline.txt    124
3 -rwxr-xr-x config.txt     642
4 -rwxr-xr-x Image         37,157,376
5 -rwxr-xr-x ramdisk.img    1,544,522
6 -rwxr-xr-x u-boot.bin     708,720

```

The `boot.scr` size (1 499 bytes) and `u-boot.bin` size (708 720 bytes) are characteristic of the non-AVB variants; both are smaller than their signed counterparts (see TC-S-002), as neither the AVB boot script nor `libavb` are compiled in.

TC-U-003 — AVB verification absent from unsigned boot path. The U-Boot serial output in TC-U-001 contains no occurrences of `avb init`, `avb verify`, `Initializing AVB`, `Verifying partitions`, or `AVB:.` Execution proceeds directly from `Loading kernel Image...` to `Booting Android...` to `booti`, confirming that the unsigned image never invokes the AVB verification path and provides no cryptographic boot-integrity check.

TC-U-004 — First-stage init reports absent vbmeta partition (negative). The unsigned image uses an MBR partition layout with no p8 (vbmeta) partition. Android’s first-stage init attempted to locate vbmeta and reported its absence:

Listing 5.4: TC-U-004: First-stage init reporting absent vbmeta partition

```

1 [ 4.126439] init: Failed to read vbmeta partitions.

```

This confirms that first-stage init correctly identifies the absence of vbmeta and that the unsigned image carries no AVB integrity guarantees.

TC-U-005 — Boot diagnostics service reports unsigned security state. The `rpi5_boot_diag` service fires on `sys.boot_completed=1` and prints a structured security summary to the UART console. On the unsigned build it produced:

Listing 5.5: TC-U-005: `rpi5_boot_diag.sh` output — unsigned build

```

1 rpi5 boot diagnostics Sun May 3 06:08:12 GMT 2026
2 =====
3
4 --- verified boot ---
5 orange
6 unlocked
7 1.1
8
9 --- global crypto state ---
10 unsupported
11

```

```

12 --- selinux ---
13 Permissive

```

The state `orange/unlocked` is the expected verified boot state for a build without a locked bootloader. `crypto state: unsupported` confirms no FDE/FBE is active. `SELinux: Permissive` matches the unsigned build configuration.

Table 5.1: Unsigned build validation results

TC ID	Type	Description	Result
TC-U-001	Functional	Unsigned image boots to Android without AVB	✓Pass
TC-U-002	Functional	Boot partition contains correct files (non-AVB variants)	✓Pass
TC-U-003	Security	No AVB commands present on unsigned boot path	✓Pass
TC-U-004	Security	First-stage init reports absent vbmeta partition	✓Pass
TC-U-005	Functional	Boot diagnostics confirms unsigned security state	✓Pass

5.3.2 Signed Build Validation

The signed build uses an AVB-enabled U-Boot, a GPT partition layout with a dedicated vbmeta partition (p8), `fail_closed` policy, and `androidboot.selinux=enforcing`. Seven test cases validate its security behaviour, including two negative cases that confirm fail-closed rejection.

TC-S-001 — Signed image boots with AVB verification passed. With the PL011-AXI patch applied, U-Boot produced serial output on power-on. The AVB boot script ran, verified the vbmeta signature, and transferred control to Android:

Listing 5.6: TC-S-001: U-Boot AVB boot script output — signed build (abbreviated)

```

1 === Android U-Boot Boot Script (AVB) ===
2 Setting AVB policy to default: fail_closed
3 AVB fail policy: fail_closed
4 Initializing AVB...
5 Verifying partitions...
6 ## Android Verified Boot 2.0 version 1.1.0
7 Verification passed successfully
8 AVB: Verification PASSED
9 Loading kernel Image...
10 Loading ramdisk...
11 bootargs: root=/dev/ram0 rootwait androidboot.hardware=rpi5

```

```

12  androidboot.selinux=permissive androidboot.encryption_mode=fbe
13  androidboot.vbmeta.device=PARTUUID=66680148-de99-4703-8f30-76949
    b3cec9a
14  androidboot.vbmeta.avb_version=1.1
15  androidboot.vbmeta.device_state=unlocked
16  androidboot.vbmeta.hash_alg=sha256
17  androidboot.vbmeta.digest=68736
    b3481643cfadb7d69022e6ae6a59370236695b3fe64c14daf87ee36f86e
18  androidboot.veritymode=enforcing
19  androidboot.verifiedbootstate=orange
20  Booting Android with AVB verification...

```

`androidboot.verifiedbootstate=orange` indicates that AVB signature verification passed but the bootloader is in unlocked state — the expected outcome on RPi5, which has no OEM fuse-locked key store [4]. `androidboot.veritymode=enforcing` confirms that dm-verity will enforce partition integrity at runtime.

TC-S-002 — Boot partition contains correct files for signed image. The FAT32 boot partition (p1) of the signed image was inspected on the host. The AVB boot script was present and both `boot.scr` and `u-boot.bin` were larger than their unsigned counterparts, reflecting the AVB commands and embedded `libavb` respectively:

Listing 5.7: TC-S-002: Boot partition file listing — signed image

```

1  -rwxr-xr-x  boot.scr          3,289  (AVB variant -- larger than unsigned
    1,499)
2  -rwxr-xr-x  cmdline.txt       176
3  -rwxr-xr-x  config.txt        642
4  -rwxr-xr-x  Image             37,157,376
5  -rwxr-xr-x  ramdisk.img       1,326,049
6  -rwxr-xr-x  u-boot.bin        769,928  (AVB-enabled -- larger than
    unsigned 708,720)

```

The size increase in `u-boot.bin` (769 928 vs 708 720 bytes) reflects the inclusion of `libavb` and the embedded RSA-4096 public key. The size increase in `boot.scr` (3 289 vs 1 499 bytes) reflects the AVB `avb init/avb verify` command sequence.

TC-S-003 — Kernel boot-layer markers confirm full boot chain. Four custom diagnostic markers were visible in the serial log, confirming each stage of the boot chain from kernel entry to userspace handoff:

Listing 5.8: TC-S-003: Kernel RPi5-DIAG boot-layer markers (abbreviated)

```

1  [ 0.000000][ T0] RPi5-DIAG: Layer 1 start_kernel entry (bootloader
    handoff complete)
2  [ 0.000000][ T0] RPi5-DIAG: Layer 2 console_init complete (early
    console up, mm init done)

```

```

3 [ 0.009196][ T1] RPi5-DIAG: Layer 3 do_initcalls entry (driver
   probe sequence begins)
4 [ 22.617616][ T1] RPi5-DIAG: Layer 4 kernel_init userspace exec (
   all drivers probed, exec /init)

```

Layer 4 appears at approximately 22.6 seconds from kernel entry, reflecting the time for all driver probes and module initialisations on the BCM2712.

TC-S-004 — dm-verity active on system and vendor partitions. First-stage init mounted `/system` and `/vendor` through device-mapper rather than directly from the raw block devices, confirming dm-verity is active on both partitions:

Listing 5.9: TC-S-004: First-stage init dm-verity mount messages (abbreviated)

```

1 [ 22.698811][ T1] init: [libfs_mgr] check_fs():
2   mount(/dev/block/by-name/metadata,/metadata,ext4)=0: Success
3 [ 22.815684][ T1] EXT4-fs (dm-0): mounted filesystem
4   d392d8bb-6c67-58f7-b0b2-091ad46c5bc2 ro without journal. Quota mode:
   none.
5 [ 22.854598][ T1] EXT4-fs (dm-1): mounted filesystem
6   2b96c597-1e2f-5ee1-9851-c4a9fa9de36e ro without journal. Quota mode:
   none.

```

The `dm-0` and `dm-1` device names confirm that `/system` and `/vendor` are served through device-mapper rather than directly from `mmcblk0p5/p6`.

TC-S-005 — FBE initialisation on signed build (partial). On the signed build with FBE enabled (`RPI5_ENABLE_FBE=true`), AVB verification passed but `vold` failed to initialise the FBE key directory before `sys.boot_completed=1` was reached, causing a reboot loop at approximately 9 seconds:

Listing 5.10: TC-S-005: FBE boot loop on signed build

```

1 AVB: Verification PASSED
2 ...
3 [ 2.893300][ T350] vold: Failed to create /metadata/vold: Permission
   denied
4 [ 2.900410][ T350] vold: read_key failed in mountFstab
5 [ 9.306520][ T1] reboot: Restarting system with command '
   bootloader '

```

The root cause (unlabelled `metadata` partition root inode under SELinux Enforcing mode) and its fix (`e2fsdroid` replacing bare `mkfs.ext4` in the SD card stage) are documented in Section 4.8. End-to-end FBE validation could not be completed within the project timeline.

TC-S-006 — Wrong public key rejected by fail-closed policy (negative). A U-Boot binary compiled with a different AVB root public key was placed on a signed SD card, with the signed vbmeta and partition images left unchanged. This scenario simulates a firmware-substitution attack. libavb detected the key mismatch and U-Boot reported:

Listing 5.11: TC-S-006: U-Boot wrong-key rejection

```

1 avb_slot_verify.c:837: ERROR: vbmeta: Public key used to sign data
  rejected.
2 Verification failed, reason: Public keys are not accepted
3 AVB: Verification FAILED
4 Fail-closed policy: resetting
5 resetting ...

```

The device reset immediately without loading the kernel, confirming that the fail-closed policy prevents boot when the trust anchor does not match.

TC-S-007 — Corrupt vbmeta header rejected by fail-closed policy (negative). The first 256 bytes of the vbmeta partition (p8) were zeroed, destroying the AVB0 magic. The rejection occurred at the header-parse stage, earlier in the verification flow than the key-check rejection in TC-S-006. U-Boot reported:

Listing 5.12: TC-S-007: U-Boot vbmeta header rejection

```

1 avb_vbmeta_image.c:46: ERROR: Magic is incorrect.
2 avb_slot_verify.c:745: ERROR: vbmeta: Error verifying vbmeta image:
  invalid vbmeta header
3 Verification failed, reason: Metadata is invalid or inconsistent
4 AVB: Verification FAILED
5 Fail-closed policy: resetting
6 resetting ...

```

The device reset immediately without loading the kernel, confirming that vbmeta integrity is enforced before any signature or key check is reached.

5.4 Evaluation Summary

Table 5.3 summarises the evaluation against the objectives defined in Chapter 1.

5.5 Limitations

- **FBE post-fix capture:** The e2fsdroid fix that resolves the metadata SELinux boot loop (Section 4.8) is implemented in the build pipeline, however the validation is pending.

Table 5.2: Signed build validation results

TC ID	Type	Description	Result
TC-S-001	Functional	Signed image boots with AVB verification passed	✓ Pass
TC-S-002	Functional	Boot partition contains correct files (AVB variants)	✓ Pass
TC-S-003	Functional	Kernel boot-layer markers confirm full boot chain	✓ Pass
TC-S-004	Security	dm-verity active on system and vendor partitions	✓ Pass
TC-S-005	Security	FBE initialisation on signed build	~ Partial
TC-S-006	Security	Wrong public key rejected by fail-closed policy	✓ Pass
TC-S-007	Security	Corrupt vbmeta header rejected by fail-closed policy	✓ Pass

Table 5.3: Evaluation against project objectives

Objective	Status	Evidence
AVB 2.0 evaluation	✓ Pass	<code>verifiedbootstate=orange; veritymode=enforcing;</code> fail-closed policy confirmed (TC-S-006, TC-S-007)
SELinux evaluation	~ Partial	Enforcing active (Android 16); boot loop resolved; full policy audit pending
FBE 2.0 evaluation	~ Partial	FBE configured and boot loop diagnosed (TC-S-005); end-to-end operation not confirmed within project timeline
Reproducible pipeline	✓ Pass	<code>rpi5-build</code> CLI
TARA	✓ Pass	Chapter 6: 23 threats, 10 residual risks

- **SELinux policy:** SELinux evaluation revealed that Android 16 transitions to Enforcing mode unconditionally at policy load time, regardless of the `androidboot.selinux=permissive` kernel argument. The resulting boot loop was diagnosed and resolved (Section 4.8), confirming basic policy compatibility. However, a full device-specific audit to confirm no residual AVC denials remain in steady state is required before treating Enforcing mode as fully validated [19].
- **Hardware-backed keystore:** The RPi5 has no TEE or dedicated security chip. FBE key material is software-protected, which is weaker against a fully privileged attacker with physical access [17].
- **OTA updates:** The pipeline produces a full SD card image. Incremental OTA support (A/B partitions, `update_engine`) is not implemented.
- **Rollback protection:** AVB rollback index counters require tamper-evident storage (eMMC RPMB or TrustZone), neither of which is available on the RPi5 in this

configuration. Rollback index values are fixed at zero.

Chapter 6

THREAT ANALYSIS AND RISK ASSESSMENT

6.1 Overview

This chapter presents a Threat Analysis and Risk Assessment (TARA) for the `meta-rpi5-secure-aosp` platform, conducted in accordance with ISO/SAE 21434 [5]. The TARA covers the full boot chain (RPi firmware through Android userspace), all implemented security mechanisms (AVB, dm-verity, FBE, SELinux), and the following attack surfaces: SD card physical write access, UART serial console, USB/fastboot interface, U-Boot interactive console, and the build pipeline.

The TARA methodology is described in Section 2.11. All scores use the SFOP impact scale (Table 2.2) and the attack potential feasibility scale defined in ISO/SAE 21434 Clause 15.

Scope note: OTA updates, network-layer threats, and physical side-channel attacks (power analysis, EM probing) are out of scope. The threat model assumes a development/research device with no safety-of-life operational path; Safety impact is therefore capped at 2 across all scenarios.

6.2 Asset Inventory

Twenty-four platform assets were identified across six categories. Table 6.1 provides the category summary.

The most security-sensitive assets are: **A-CRED-001** (AVB private signing key) — compromise allows signing arbitrary images that pass AVB verification; **A-IMG-004** (`vbmeta.img`) — the root of the AVB trust chain; **A-IMG-005** (U-Boot binary) — the first component in the chain that performs cryptographic verification; and **A-CFG-001** (`boot_avb.scr`) — currently unsigned, hence replaceable without breaking the AVB

Table 6.1: Asset inventory summary by category (24 assets total)

Category	Name	Count	Representative assets
CRED	Credentials	3	AVB private key, U-Boot compiled public key, FBE keys
IMAGE	Partition images	6	<code>boot.img</code> , <code>system.img</code> , <code>vendor.img</code> , <code>vbmeta.img</code> , U-Boot binary, RPi firmware
CONFIG	Configuration	4	<code>boot_avb.scr</code> , SD card YAML, <code>fstab.rpi5.*</code> , build config YAML
IFACE	Interfaces	4	UART console, USB/fastboot, U-Boot interactive console, SD card slot
BUILD	Build pipeline	4	Build orchestrator, patch stack, build host, <code>avbtool</code>
DATA	Runtime data	3	<code>/data</code> (userdata), <code>/metadata</code> , U-Boot env vars

chain.

6.3 Threat Scenarios

The STRIDE model [20] was applied to each asset category. Table 6.3 lists all 23 threat scenarios with their STRIDE category, affected asset, and attack vector.

Table 6.3: Threat scenarios (23 total)

ID	STRIDE	Threat summary	Asset	Attack vector
T-001	T	Replace RPi firmware blobs to bypass AVB entirely	A-IMG-006	Physical (SD card)
T-002	T	Replace U-Boot binary with attacker-key-embedded binary	A-IMG-005	Physical (SD card)
T-003	T	Tamper <code>boot.img</code> (kernel/ramdisk) on SD card	A-IMG-001	Physical (SD card)
T-004	T	Tamper <code>system.img</code> / <code>vendor.img</code> blocks after signing	A-IMG-002/003	Physical (SD card)
T-005	T	Replace <code>vbmeta</code> with attacker-signed version	A-IMG-004	Physical (SD card)
T-006	S	Full spoofing: replace <code>vbmeta</code> + U-Boot to appear orange-state	A-IMG-004/005	Physical (SD card)

Table 6.3 (continued)

ID	STRIDE	Threat summary	Asset	Attack vector
T-007	E	U-Boot console via UART — arbitrary partition write or env change	A-IFACE-003	Physical (UART)
T-008	T	Replace <code>boot_avb.scr</code> to disable AVB result check	A-CFG-001	Physical (SD card)
T-009	T	Inject malicious patch into <code>patches/</code> repository	A-BUILD-002	Build pipeline
T-010	T	Modify stage script to skip signing or swap key	A-BUILD-001	Build pipeline
T-011	T	Replace <code>avbtool</code> binary with attacker-controlled version	A-BUILD-004	Build pipeline
T-012	I	Read AVB private key from build host <code>config/avb/</code>	A-CRED-001	Build pipeline
T-013	R	No audit trail linking signed artifact to source commit	A-BUILD-001	Build pipeline
T-014	I	UART log monitoring — extract boot config and partition info	A-IFACE-001	Physical (UART)
T-015	T	USB fastboot partition flashing without OEM lock	A-IFACE-002	Physical (USB)
T-016	D	Physical removal of SD card — DoS	A-IFACE-004	Physical (SD slot)
T-017	D	Corrupt <code>vbmeta</code> partition — permanent boot failure (fail-closed)	A-IMG-004	Physical (SD card)
T-018	I	Read plaintext userdata from SD card (no FBE, or FBE keys extracted)	A-DATA-001	Physical (SD card)
T-019	I	Extract FBE keys from kernel memory at runtime	A-CRED-003	Local (kernel exec)
T-020	D	Corrupt <code>/metadata</code> partition — FBE failure or factory reset	A-DATA-002	Physical (SD card)
T-021	T	Modify build config YAML to silently disable signing or set fail-open	A-CFG-004	Build pipeline

Table 6.3 (continued)

ID	STRIDE	Threat summary	Asset	Attack vector
T-022	T	Remove FBE flags from <code>fstab</code> — silent encryption disable	A-CFG-003	Build pipeline
T-023	E	Overwrite U-Boot environment variables to redirect boot	A-DATA-003	Physical (SD card)

6.4 Risk Register

Risk Level = Impact $\times$ Feasibility, classified as: Low (1–4), Medium (5–8), High (9–12), Critical (13–16). Tables 6.4–6.12 give the full risk register by category.

Table 6.4: Risk register: Boot chain threats (T-001 to T-008)

ID	Threat summary	Impact	Feasibility	Score	Level
T-001	Replace RPi firmware to bypass AVB	O:4	4	16	Critical
T-002	Replace U-Boot with attacker key	O:4	3	12	High
T-003	Tamper <code>boot.img</code>	O:4	4	16	Critical
T-004	Tamper <code>system/vendor.img</code>	O:3	3	9	High
T-005	Replace <code>vbmeta</code>	O:3	3	9	High
T-006	Full spoofing (<code>vbmeta</code> + U-Boot)	O:4	2	8	Medium
T-007	U-Boot console access via UART	O:4	4	16	Critical
T-008	Replace <code>boot_avb.scr</code>	O:4	4	16	Critical

The concentration of Critical risks in the boot chain and physical interface categories reflects the dominant exposure of the platform: all security-critical content resides on a removable SD card with no hardware write protection, accessible to any attacker with physical access and standard PC tools. T-019 scores 4 (Low) because extracting FBE keys from a running kernel requires prior kernel code execution — an extreme prerequisite.

Table 6.6: Risk register: Build pipeline threats (T-009 to T-013)

ID	Threat summary	Impact	Feasibility	Score	Level
T-009	Inject malicious patch	O:4	2	8	Medium
T-010	Modify stage script	O:4	2	8	Medium
T-011	Replace <code>avbtool</code>	O:4	2	8	Medium
T-012	Read AVB private key	P:3	3	9	High
T-013	No signing audit trail	O:2	4	8	Medium

Table 6.8: Risk register: Runtime interface threats (T-014 to T-017)

ID	Threat summary	Impact	Feasibility	Score	Level
T-014	UART log monitoring	P:2	4	8	Medium
T-015	USB fastboot without OEM lock	O:4	4	16	Critical
T-016	Physical SD card removal (DoS)	O:4	4	16	Critical
T-017	Corrupt vbmeta — fail-closed DoS	O:4	4	16	Critical

6.5 Mitigation Mapping

Table 6.16 maps every Critical and High risk to its countermeasure and implementation status. The status legend is: **Impl.** = implemented and active in current build; **Partial** = implemented but not yet validated on hardware; **Planned** = identified in the backlog, not yet implemented; **Not Started** = no countermeasure exists.

Table 6.16: Mitigation mapping for Critical and High risks

ID	Countermeasure	Evidence / Pointer	Status
T-001	Physical security; RPi5 EEPROM secure boot (future)	No software countermeasure available; EEPROM path in §7.4	Not Started
T-002	EEPROM secure boot to verify U-Boot load	Same as T-001	Not Started

Table 6.16 (continued)

ID	Countermeasure	Evidence / Pointer	Status
T-003	AVB hash footer on boot.img; fail-closed reset on mismatch	stages/sign.py; boot_avb.cmd	Partial
T-004	AVB hashtree footer + dm-verity on system/vendor	stages/sign.py; avbtool add_hashtree_footer	Impl.
T-005	vbmata.img signed with RSA-4096; compiled public key in U-Boot	stages/sign.py; stages/build.py	Impl.
T-007	U-Boot console lockdown (CONFIG_BOOTDELAY=0, CONFIG_DISABLE_CONSOLE)	Planned production U-Boot config	Planned
T-008	FIT-image signed script (U-Boot FIT signing)	Not yet implemented	Not Started
T-012	Build host filesystem permissions; key excluded from git	.gitignore; access control policy	Partial
T-015	Remove CONFIG_CMD_FASTBOOT in production U-Boot build	Planned production defconfig variant	Planned
T-016	Physical enclosure or migration to eMMC (future)	Deployment recommendation	Not Started
T-017	A/B slot fallback (OTA workstream); re-flash recovery	Planned (P3 OTA workstream)	Planned
T-018	FBE (fscrypt v2, AES-256-XTS); metadata encryption	fstab.rpi5.fbe; stages/sdcard.py	Partial
T-020	e2fsdroid SELinux labelling ensures vold can access /metadata	stages/sdcard.py; Section 4.8	Impl.
T-023	CONFIG_ENV_IS_NOWHERE for production builds	Planned production U-Boot config	Planned

Table 6.10: Risk register: Data confidentiality threats (T-018 to T-020)

ID	Threat summary	Impact	FeasibilityScore	Level	
T-018	Read plaintext userdata from SD card	P:4	4	16	Critical
T-019	Extract FBE keys from kernel memory (runtime)	P:4	1	4	Low
T-020	Corrupt /metadata — FBE failure	O:3	4	12	High

Table 6.12: Risk register: Configuration and policy threats (T-021 to T-023)

ID	Threat summary	Impact	FeasibilityScore	Level	
T-021	Modify build YAML to disable signing	O:3	2	6	Medium
T-022	Remove FBE flags from fstab	P:3	2	6	Medium
T-023	Overwrite U-Boot environment vars	O:4	3	12	High

6.6 Residual Risk Register

Ten risks remain after mitigation. Table 6.17 records the residual risk level, acceptance category, and follow-up actions for each. The acceptance categories are: *Platform Constraint* (inherent hardware limitation, cannot be resolved without a different SoC); *Deferred* (mitigatable but not yet implemented); *Operational* (mitigated by physical or process controls outside the software scope); *Research* (acceptable for a research device, not for production).

Table 6.14: Risk level summary across all 23 threat scenarios

Risk Level	Count	Threat IDs
Critical (13–16)	9	T-001, T-003, T-007, T-008, T-015, T-016, T-017, T-018
High (9–12)	6	T-002, T-004, T-005, T-012, T-020, T-023
Medium (5–8)	7	T-006, T-009, T-010, T-011, T-013, T-014, T-021, T-022
Low (1–4)	1	T-019

Table 6.17: Residual risk register (10 accepted risks)

ID	Residual risk statement	Level	Category	Production mitigation path
RR-001	RPi firmware not in AVB chain; attacker can replace it	High	Platform Constraint	RPi5 EEPROM secure boot
RR-002	U-Boot console reachable via UART (dev builds)	High	Deferred	CONFIG_BOOTDELAY=0 + locked build
RR-003	boot_avb.scr unsigned on FAT32	High	Deferred	U-Boot FIT image signing
RR-004	USB fastboot without OEM lock (dev builds)	Critical	Deferred	Remove CONFIG_CMD_FASTBOOT
RR-005	Physical SD card removal (DoS)	Medium	Platform / Operational	Locked enclosure or eMMC
RR-006	vbmata corruption causes permanent boot failure (no A/B)	Medium	Deferred	A/B slot + update_engine
RR-007	FBE keys software-protected only — no TEE on RPi5	Medium	Platform Constraint	Platform with hardware TEE (ex., OP-TEE)
RR-008	U-Boot binary itself not verified by a higher authority	High	Platform Constraint	RPi5 EEPROM secure boot
RR-009	AVB private key on build host filesystem (dev key)	Medium	Research	HSM or secrets management service
RR-010	U-Boot environment variables writable from SD card	High	Deferred	CONFIG_ENV_IS_NOWHERE

The six “Platform Constraint” and “Platform Constraint / Deferred” residual risks (RR-001, RR-005, RR-007, RR-008) are not implementation failures — they reflect the security ceiling of the RPi5 hardware as used in this project. The remaining four (RR-002, RR-003, RR-004, RR-010) are deferred gaps that are addressable within the current platform.

6.7 Discussion: Software Security Ceiling on RPi5

The risk profile reveals that the dominant structural exposure is physical access to an unprotected removable SD card. This single attack surface accounts for the majority of Critical-rated threats. Software controls — AVB signatures, dm-verity, FBE, fail-closed policy — detect or prevent tampering once U-Boot is executing, but they cannot prevent a physical attacker from bypassing U-Boot itself (RR-001, RR-008) or from replacing the unsigned boot script before U-Boot runs (RR-003).

A second structural limit is the absence of a hardware TEE. FBE provides meaningful protection against offline SD card reads (T-018 is partially mitigated) but does not

protect against an attacker who can execute code in kernel context on a running device. Production deployments would require a platform that exposes TrustZone to software (via OP-TEE [23]) so that FBE master keys are hardware-bound and non-extractable, and a hardware keystore (Keymaster/KeyMint [24]) to wrap keys under TEE-held material.

Comparing the platform’s posture to a production automotive ECU: a production device would additionally carry RPi5 EEPROM secure boot (or equivalent SoC-level root of trust), a hardware TEE with Keymaster, signed boot scripts, locked U-Boot console, disabled fastboot, eMMC storage (non-removable), and A/B OTA update support. Each of these mitigates one or more residual risks from Table 6.17.

6.8 Technical Reasoning

Causal chain: The concentration of Critical risks in the boot chain and physical interface categories (T-001, T-003, T-007, T-008, T-015–T-018) reflects the fundamental exposure of a commodity SBC: all security-critical content is on a removable medium with no hardware write protection. The Impact score is driven primarily by Operational loss (ability to subvert boot) and Privacy (userdata exposure), not Safety — consistent with a research device with no safety-of-life operational context.

Why the SFOP model is appropriate: An alternative such as the EVITA threat model uses a 5-level impact scale and quantifies financial impact in monetary terms. The 4-level ISO/SAE 21434 SFOP scale was selected because it is the mandated methodology for automotive cybersecurity and produces sufficient differentiation between risk tiers on this platform without introducing spurious precision.

Validity of scores: Feasibility scores intentionally reflect the development posture (UART unlocked, fastboot enabled, no OEM lock). In a production hardening configuration, locking the U-Boot console, disabling fastboot, and enforcing OEM lock would drop several Critical risks to Medium or High. Scores were not deflated to minimise the apparent risk profile; the Critical count (9) accurately reflects an unlocked development build.

Value of TARA before hardware validation: Conducting the TARA before completing all hardware validation tests (FBE TC-FBE-001/002 is pending) revealed the unsigned boot script gap (RR-003) and the writable U-Boot environment (RR-010) as High-risk issues that were not visible from the implementation perspective alone. These are now tracked as explicit follow-up items rather than implicit design debt.

6.9 External References

- ISO/SAE 21434 TARA methodology, SFOP impact categories, and residual risk acceptance requirements [5].

- STRIDE threat model categories [20].
- OP-TEE as the mitigation path for software-only FBE key protection (RR-007) [23].
- Android hardware-backed Keystore as the production alternative to software-only key management [24].
- RPi5 EEPROM secure boot as the path to extend the trust chain below U-Boot (RR-001, RR-008) [25].
- Android FBE key hierarchy and the role of `vold` in key management [16, 17].

Chapter 7

CONCLUSION

7.1 Summary

This project evaluated the feasibility of running a security-hardened Android Automotive OS on commodity Raspberry Pi 5 hardware by enabling and assessing three Android security subsystems: Android Verified Boot 2.0 (AVB2), SELinux, and File-Based Encryption 2.0 (FBE). To support reproducible evaluation, the `meta-rpi5-secure-aosp` framework as developed, integrating U-Boot, the AVB signing pipeline, FBE, and per-boot security diagnostics into a single orchestrated build pipeline.

The key engineering contributions are:

1. **U-Boot PL011-AXI patch:** Resolved the fundamental serial console bring-up issue on RPi5 by adding the `"arm,pl011-axi"` compatible string to the PL011 driver match table, enabling all subsequent UART-based debugging [8, 10].
2. **External AVB signing pipeline:** Implemented a clean separation between the AOSP build and the signing step, preserving private key secrecy and allowing the same images to be signed with different keys without rebuilding [4].
3. **FBE on RPi5:** Implemented file-based encryption configuration on `userdata` (AES-256-XTS) with metadata encryption (`/metadata`), building on a three-way fstab selection mechanism that preserves backward compatibility with unsigned builds [16]. End-to-end operation could not be confirmed on hardware within the project timeline (Section 4.8).
4. **FBE SELinux bring-up fix:** Identified and resolved the concurrent SELinux enforcing behaviour and unlabelled metadata partition root inode that caused a reboot loop, by replacing `mkfs.ext4` with `e2fsdroid` in the SD card stage (Section 4.8).
5. **Boot diagnostics:** Provided a lightweight, ADB-free mechanism for verifying the security state of the device at every boot via both kernel-level diagnostic markers

and a post-boot diagnostic shell script.

6. **Build orchestrator:** Delivered a well-structured Python CLI with a shared context model, idempotent patch application, and guards that reject non-safe boot-state and AVB policy combinations unless explicitly overridden.
7. **TARA (ISO/SAE 21434):** Conducted a full Threat Analysis and Risk Assessment over the platform, producing a risk register of 23 threat scenarios (9 Critical, 6 High, 8 Medium, 1 Low), a mitigation mapping for all High and Critical risks, and an explicit residual risk register of 10 accepted risks with rationale and production hardening paths (Chapter 6) [5].

Evaluation on real RPi5 hardware confirmed that the AVB objective was met: `vbmeta` verification passes (`verifiedbootstate=orange`, `veritymode=enforcing`), and the fail-closed AVB policy correctly halts the device when images are tampered with. SELinux evaluation revealed that Android 16 enforces Enforcing mode unconditionally; the resulting boot loop was diagnosed and resolved, though a full device-specific policy audit remains as future work. FBE was configured and the boot loop root cause was identified and fixed, but end-to-end FBE operation could not be confirmed on hardware within the project timeline. The TARA quantifies the residual security posture and identifies the hardware prerequisites for production deployment.

Note on verified boot state: The reported `verifiedbootstate=orange` reflects that the `vbmeta.img` signature is cryptographically valid and `system/vendor` hashtrees pass dm-verity, but the bootloader is in unlocked state because the RPi5 has no OEM fuse-locked key store. This is the expected outcome for this platform. Additionally, the current implementation does *not* include `boot.img` in the `vbmeta` descriptor chain — the boot partition is loaded directly via `fatload` without hash verification. A truly complete verified boot chain requires partition re-architecture to bring `boot.img` into the AVB chain (Section 7.4).

7.2 Security Posture Assessment

The TARA (Chapter 6) reveals that the platform’s security ceiling is set by two structural constraints of the RPi5:

1. **Physical SD card access:** All partitions reside on a removable medium with no hardware write protection. Software-enforced controls (AVB, dm-verity, fail-closed policy) detect tampering once U-Boot executes, but cannot prevent an attacker from bypassing U-Boot itself by replacing the firmware or the boot script before it runs.

2. **Absent hardware TEE:** FBE master keys are protected in software only. Hardware-backed key storage via a TEE (Keymaster/KeyMint [24]) would prevent key extraction even from a fully compromised running system.

The four deferred residual risks (RR-002, RR-003, RR-004, RR-010) are addressable within the RPi5 platform by locking the U-Boot console, signing the boot script with FIT image verification, disabling fastboot, and setting `CONFIG_ENV_IS_NOWHERE`. These represent planned production hardening steps that were deferred to preserve development convenience.

7.3 Technical Reasoning

7.3.1 Causal Chain

The absence of "arm,pl011-axi" in U-Boot's DM match table is the root cause of the serial console failure [22]. The causal chain is: BCM2712 firmware DT → `stdout-path` = `serial10` → `compatible` = `arm,pl011-axi` → U-Boot DM bind failure → no UART output. All downstream debugging and AVB bring-up depended on resolving this single compatibility gap first.

The FBE boot loop root cause is a cross-layer failure: a MAC policy enforcement decision (SELinux enforcing despite cmdline permissive) combined with a toolchain gap (bare `mkfs.ext4` not applying SELinux labels) produced a failure in the encryption subsystem (void key initialisation). Debugging required correlating evidence from all three layers simultaneously.

7.3.2 Alternatives Considered

An alternative to patching U-Boot would have been to use a Device Tree overlay to replace "arm,pl011-axi" with "arm,pl011" in the firmware DT. This was rejected because: (a) the RPi firmware DT isn't easily overridden at U-Boot load time without additional scripts; and (b) modifying the upstream DT obscures the actual hardware description. The driver-side fix is the correct and upstreamable approach.

For the signing pipeline, an alternative was to use `BOARD_AVB_ENABLE` inside the AOSP build. This was rejected because it requires the private key to be accessible within the build environment, which contradicts the security separation goal and the standard automotive production signing workflow.

7.3.3 Risk and Tradeoffs

The primary security limitation is the lack of hardware-backed key storage. On a production vehicle IVI, FBE keys would be anchored in a TEE (ex., Trusty [26]) or a dedicated

security chip (ex: Titan M). On the RPi5, the key material is software-protected by the Android Keystore, which provides meaningful protection against offline attacks but is weaker against a fully privileged attacker with physical access to the device [17].

The permissive SELinux mode was intended as a deliberate tradeoff for the current research stage. In practice, Android 16 transitions SELinux to enforcing at policy load time regardless of the command-line flag (Section 4.8), which exposed the metadata partition labelling gap. While the boot loop was resolved, a complete device-specific policy audit is required to confirm no residual AVC denials remain in steady state [19].

7.4 Future Work

- **Boot partition AVB integration (partition re-architecture):** The current implementation loads `boot.img` via `fatload` without hash verification; `vbmeta.img` contains no descriptor for `boot.img`. Achieving a fully verified boot chain requires: (1) adding an AVB hash footer to `boot.img` at sign time (`avbtool add_hash_footer`), (2) including a hash descriptor for `boot.img` in `vbmeta.img`, and (3) loading the boot partition via the U-Boot AVB command (`avb read_rollback / avbloadpart`) rather than directly via `fatload`. This change requires the kernel and ramdisk to reside on a raw GPT partition rather than a FAT32 file, which in turn requires a boot partition layout change and U-Boot boot script update.
- **Signed boot script (RR-003):** Wrap `boot_avb.scr` in a U-Boot FIT image with a verified FIT signature, extending the AVB chain to cover the boot script.
- **U-Boot console lockdown (RR-002, RR-010):** Implement a production U-Boot config variant with `CONFIG_BOOTDELAY=0`, `CONFIG_DISABLE_CONSOLE`, and `CONFIG_ENV_IS_NOWHERE`.
- **Fastboot disable (RR-004):** Create a production defconfig that removes `CONFIG_CMD_FASTBOOT` and `CONFIG_USB_GADGET`.
- **SELinux policy audit:** Android 16 already runs SELinux in Enforcing mode unconditionally (Section 4.8). A device-specific file-context and service-context audit is required to confirm that no residual AVC denials affect steady-state operation [19].
- **RPi5 EEPROM secure boot (RR-001, RR-008):** Investigate OTP key provisioning and signed FIT image boot to extend the trust chain below U-Boot [25].
- **TEE integration (RR-007):** Explore deploying OP-TEE [23] on the RPi5 to provide hardware-backed key storage for FBE master keys.
- **A/B OTA updates (RR-006):** Extend the partition layout to support A/B system partitions and integrate `update_engine` for over-the-air updates.

- **FBE hardware validation:** Confirm `ro.crypto.state=encrypted` across multiple boot cycles and validate file accessibility per DE/CE class (test cases TC-FBE-001, TC-FBE-002), updating RR-007 with measured evidence.
- **Rollback protection:** Explore software-emulated rollback index storage in a tamper-evident partition as an intermediate measure before TEE availability.
- **U-Boot upstream contribution:** Submit the `arm,pl011-axi` patch to the U-Boot mailing list for the benefit of the broader RPi5 developer community.

7.5 External References

The following are authoritative public sources directly relevant to the technical claims in this report:

- Android Verified Boot 2.0 specification and `avbtool` documentation [4].
- Android File-Based Encryption design [16].
- dm-verity implementation in Android [13].
- Android Automotive OS platform overview [3].
- ARM PrimeCell UART PL011 Technical Reference Manual (DDI0183G) [8].
- Devicetree Specification, `/chosen` node and `stdout-path` [22].
- Android SELinux device policy development [19].
- ISO/SAE 21434 cybersecurity engineering standard (TARA methodology) [5].
- STRIDE threat modelling [20].
- OP-TEE hardware TEE integration [23].
- Android hardware-backed Keystore [24].
- RPi5 EEPROM secure boot [25].
- Trusty TEE download and build guide [26].
- Quarkslab deep-dive on Android FBE and key derivation [17].

Bibliography

- [1] Android Open Source Project. Architecture Overview. <https://source.android.com/docs/core/architecture>, 2026. Official AOSP software stack: Applications, Application Framework, System Services, Android Runtime, HAL, Native Daemons, and Linux Kernel layers. Accessed: June 2026.
- [2] Android Open Source Project. Android Security Features. <https://source.android.com/docs/security/features>, 2024. Overview of Android’s multi-layered security model: application sandbox, Android Keystore, Trusty TEE, Gatekeeper/Weaver authentication, verified boot, SELinux, app signing, and encryption frameworks. Accessed: May 2026.
- [3] Android Open Source Project. What is Android Automotive? https://source.android.com/docs/automotive/start/what_automotive, 2024. Overview of Android Automotive OS (AAOS): IVI platform, AOSP-based, distinct from Android Auto. Accessed: February 2026.
- [4] Android Open Source Project. Android Verified Boot 2.0 — Official README (android16-release). <https://android.googlesource.com/platform/external/avb/+/android16-release/README.md>, 2024. Canonical reference for AVB2 VBMeta structure, rollback protection, A/B support, and BOARD_AVB_ENABLE build integration. Accessed: February 2026.
- [5] ISO/SAE. ISO/SAE 21434:2021 — Road vehicles — Cybersecurity engineering. <https://www.iso.org/standard/70918.html>, 2021. Defines TARA methodology, SFOP impact categories (Safety, Financial, Operational, Privacy), and attack potential feasibility attributes for automotive cybersecurity. Clause 15 governs the Risk Assessment phase. Accessed: May 2026.
- [6] pinout.xyz. Raspberry Pi UART Pinout. <https://pinout.xyz/pinout/uart>, 2024. GPIO 14 (TX) and GPIO 15 (RX) on the 40-pin header; 115200 baud default. Accessed: October 2025.
- [7] Raspberry Pi Ltd. Raspberry Pi Hardware Documentation. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>, 2024. Official

- hardware reference for RPi product line including BCM2712-based Raspberry Pi 5 specifications. Accessed: October 2025.
- [8] ARM Limited. ARM PrimeCell UART (PL011) Technical Reference Manual. Technical Report DDI0183G, ARM Limited, 2011. <https://developer.arm.com/documentation/ddi0183/g>. Defines the PL011 UART register map, compatible hardware variants, and AXI bus interface. Accessed: October 2025.
- [9] U-Boot Project. U-Boot: Building with GCC. <https://docs.u-boot.org/en/latest/build/gcc.html>, 2024. Covers cross-compilation setup, defconfig selection, and CROSS_COMPILE usage. Accessed: October 2025.
- [10] U-Boot Project. U-Boot Serial PL01x Driver (`drivers/serial/serial_pl01x.c`). https://source.denx.de/u-boot/u-boot/-/blob/master/drivers/serial/serial_pl01x.c, 2024. Driver implementing DM serial ops for ARM PL010/PL011 UARTs. The `udevice_id` match table maps compatible strings to `TYPE_PL011`. Accessed: October 2025.
- [11] AlenHazard et al. U-Boot for Raspberry Pi 5. Raspberry Pi Forums, <https://forums.raspberrypi.com/viewtopic.php?t=369150>, 2024. Thread initiated April 16, 2024. Discusses U-Boot v2024.04 initial RPi5 support, PCIe limitations, and serial console bring-up. Accessed: October 2025.
- [12] Raspberry Pi Forums users. Raspberry Pi 5 U-Boot Boot Issues (Splash Screen Hang). Raspberry Pi Forums, <https://forums.raspberrypi.com/viewtopic.php?t=387528>, 2024. Documents `CONFIG_BOOTDELAY=-2` fix for RPi5 U-Boot hang; recommends `bcm2711_defconfig` over `bcm2712_defconfig`. Accessed: October 2025.
- [13] Android Open Source Project. Implement dm-verity. <https://source.android.com/docs/security/features/verifiedboot/dm-verity>, 2024. SHA-256 hash tree construction, forward error correction (FEC), and verity mapping table. Android 7.0+ includes Reed-Solomon FEC. Accessed: February 2026.
- [14] jcrutchvt10. AVBTOOL — Android Verified Boot Tool (GitHub Mirror). <https://github.com/jcrutchvt10/AVBTOOL>, 2024. Mirror of AOSP `platform/external/avb`; includes `avbtool`, `libavb`, and `libavb_ab`. MIT licence. Accessed: February 2026.
- [15] Android Open Source Project. Full-Disk Encryption. <https://source.android.com/docs/security/features/encryption/full-disk>, 2024. Legacy block-level encryption using `dm-crypt` (128-bit AES-CBC / ESSIV:SHA256); deprecated from Android 10, removed in Android 13; seven-step key derivation chain involving `scrypt` and TEE-bound keys. Accessed: May 2026.

- [16] Android Open Source Project. File-Based Encryption. <https://source.android.com/docs/security/features/encryption/file-based>, 2024. Per-file encryption with independent keys; DE and CE storage areas; `fscrypt` integration. Mandatory from Android 10+. Accessed: March 2026.
- [17] Quarkslab. Android Data Encryption in Depth. <https://blog.quarkslab.com/android-data-encryption-in-depth.html>, 2023. In-depth analysis of Android FBE, CE/DE key derivation via Gatekeeper and Weaver, and PoC exploits on MediaTek and Titan M. Accessed: March–April 2026.
- [18] Android Open Source Project. Build SELinux Policy — Precompiled Policy. <https://source.android.com/docs/security/features/selinux/build#precompiled-policy>, 2024. Describes Android 8.0+ split policy architecture and precompiled policy loading via SHA256 hash comparison in `init`. Accessed: January 2026.
- [19] Android Open Source Project. Write SELinux Device Policy. <https://source.android.com/docs/security/features/selinux/device-policy>, 2024. Best practices for device bring-up SELinux policy; permissive-first workflow, service domain templates, policy size guidelines. Accessed: January 2026.
- [20] Microsoft Security Engineering. Threat Modeling Tool Threats — STRIDE. <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>, 2024. STRIDE threat category definitions: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. Accessed: May 2026.
- [21] Ganesh Halhota. Android Local Manifest for Raspberry Pi (AOSP 16). https://github.com/ganeshhalhota/android_local_manifest, 2024. Device manifest for building AOSP Android 16 on RPi4/RPi5. Includes Automotive, Tablet, and TV build targets. Accessed: October 2025.
- [22] Devicetree.org. Devicetree Specification — Chapter 3: Device Node Requirements, /chosen Node. <https://devicetree-specification.readthedocs.io/en/latest/chapter3-devicenodes.html>, 2023. Defines the `stdout-path` property used by RPi5 firmware DT to designate `serial10` as the boot console. Accessed: October 2025.
- [23] Linaro / OP-TEE Project. Open Portable Trusted Execution Environment (OP-TEE). <https://optee.readthedocs.io/en/latest/>, 2024. Reference TEE implementation for ARM TrustZone platforms; provides hardware-backed key storage for FBE master keys. Accessed: May 2026.

- [24] Android Open Source Project. Android Keystore — Hardware-backed key storage. <https://source.android.com/docs/security/features/keystore>, 2024. Hardware-backed key storage via Keymaster/KeyMint; TEE-bound key material cannot be extracted even on a rooted device. Production alternative to software-only FBE master key protection. Accessed: May 2026.
- [25] Raspberry Pi Foundation. Raspberry Pi 5 Secure Boot (EEPROM Configuration). <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#secure-boot>, 2024. Describes RPi5 EEPROM-based secure boot: OTP key provisioning and signed FIT image verification at firmware load time. The mitigation path for extending the trust chain below U-Boot. Accessed: May 2026.
- [26] Android Open Source Project. Download and Build Trusty TEE for Android. <https://source.android.com/docs/security/features/trusty/download-and-build>, 2024. Steps for building the Trusty TEE (lk.bin) for ARM64; QEMU test environment included. Accessed: February 2026.